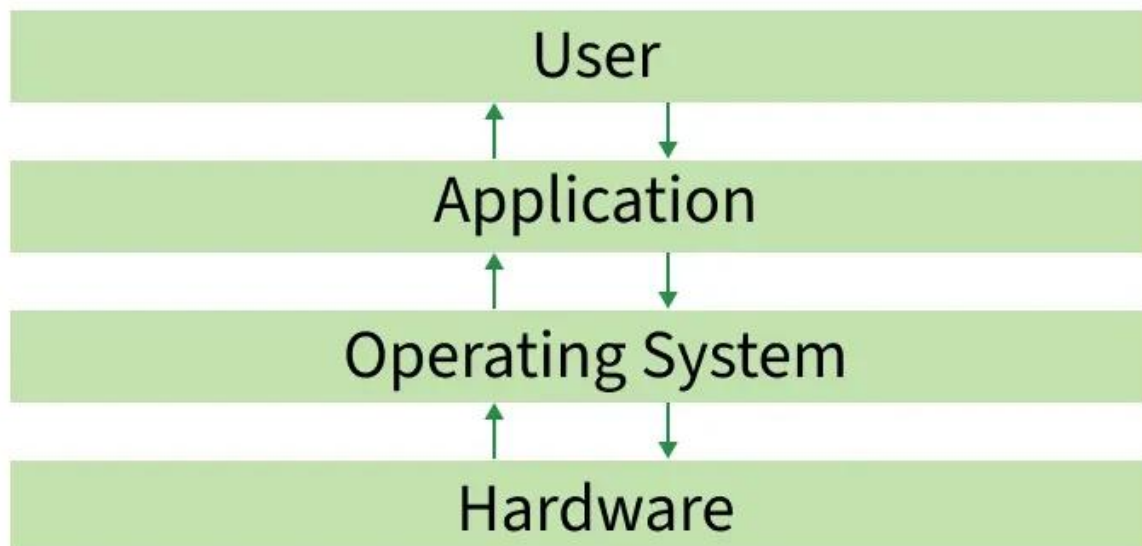


Linux/Unix Tutorial

Introduction to Operating System –

An operating system acts as an intermediary between the computer hardware and the user. In short, it is an interface between computer hardware and the user.

Control flow



List of Common Operating Systems

There are multiple types of operating systems each having its own unique features:

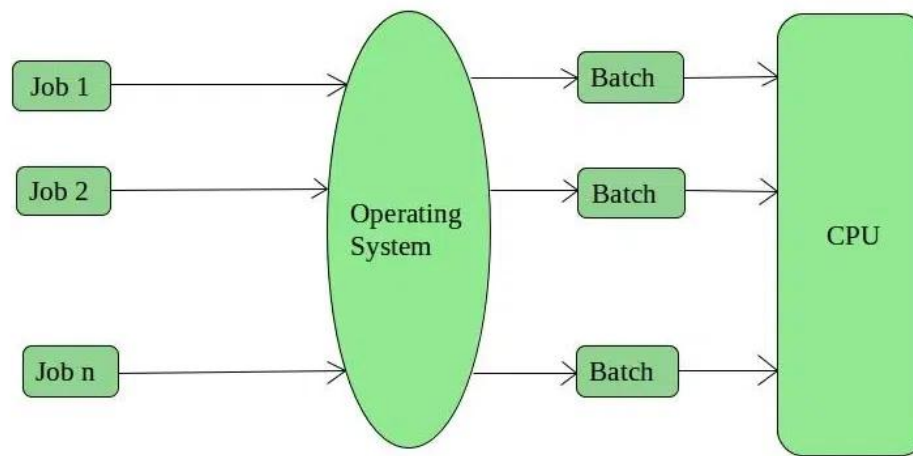
- **Windows OS:** Developed by Microsoft. Used in personal computing, business environment, gaming.
- **macOS:** Developed by Apple. Used for creative industries (design, video editing, music production), Personal computing and Professional environments.
- **Linux:** Developed by Linus Torvalds and maintained by the open-source community (Linux Foundation, Red Hat, Canonical, etc.). Used in servers and data centers, development and programming, Personal computing for tech enthusiasts.

- **Unix:** Developed originally AT&T Bell Labs, various commercial and open-source versions available. Used in servers and workstations, Development environments, Research and academic settings.

Types of Operating Systems

1. Batch Operating System

A [Batch Operating System](#) is designed to handle large groups of similar jobs efficiently. It does not interact with the computer directly but instead processes jobs that are grouped by an operator. These jobs are queued and executed one after the other, without user

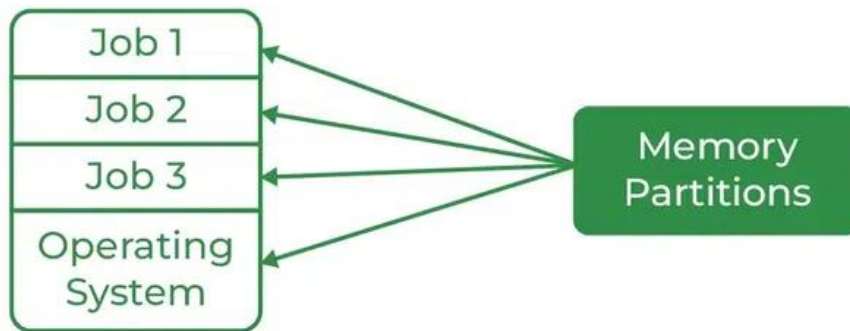


interaction during the process.

2. Multi-Programming Operating System

In a [Multi-Programming Operating System](#), multiple programs run in memory at the same time. The CPU switches between programs, utilizing its resources more effectively and

Multiprogramming

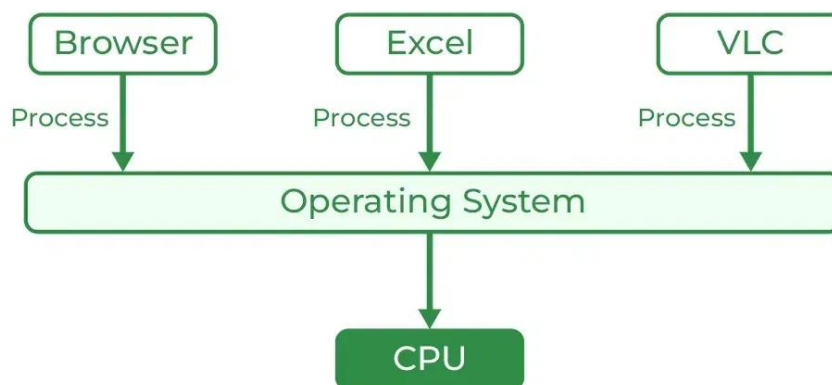


improving overall system performance.

3. Multi-tasking/Time-sharing Operating systems

[Multitasking OS](#) is a type of multiprogramming system where each process runs in a **round-robin** manner. Every task gets a fixed time slice called a **quantum**. After the quantum ends, the OS switches to the next task, allowing multiple tasks—whether from one user or many—to run smoothly on a single system.

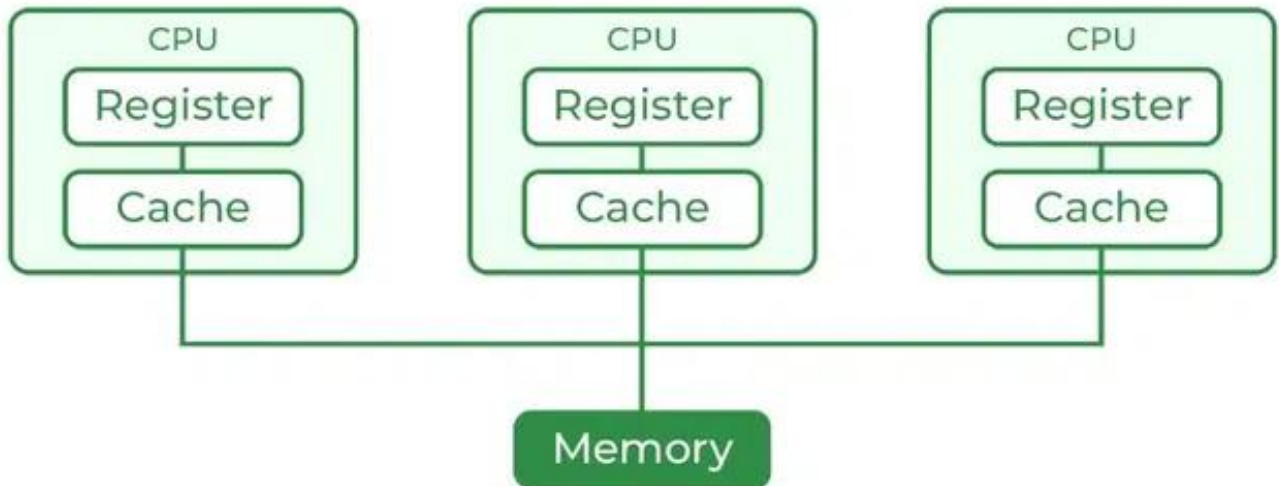
Multitasking



4. Multi-Processing Operating System

[A Multi-Processing Operating System](#) is a type of Operating System in which more than one CPU is used for the execution of resources. It better the throughput of the System.

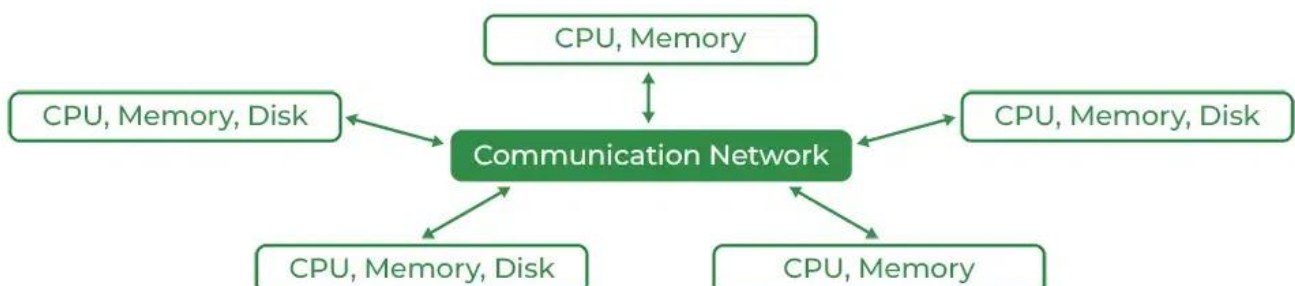
Multiprocessing



5. Distributed Operating System

[Distributed operating systems](#) connects multiple independent computers through a shared communication network. Each system has its own CPU and memory but works together as a single unit. The main benefit is remote access, allowing users to use files and software stored on other connected systems.

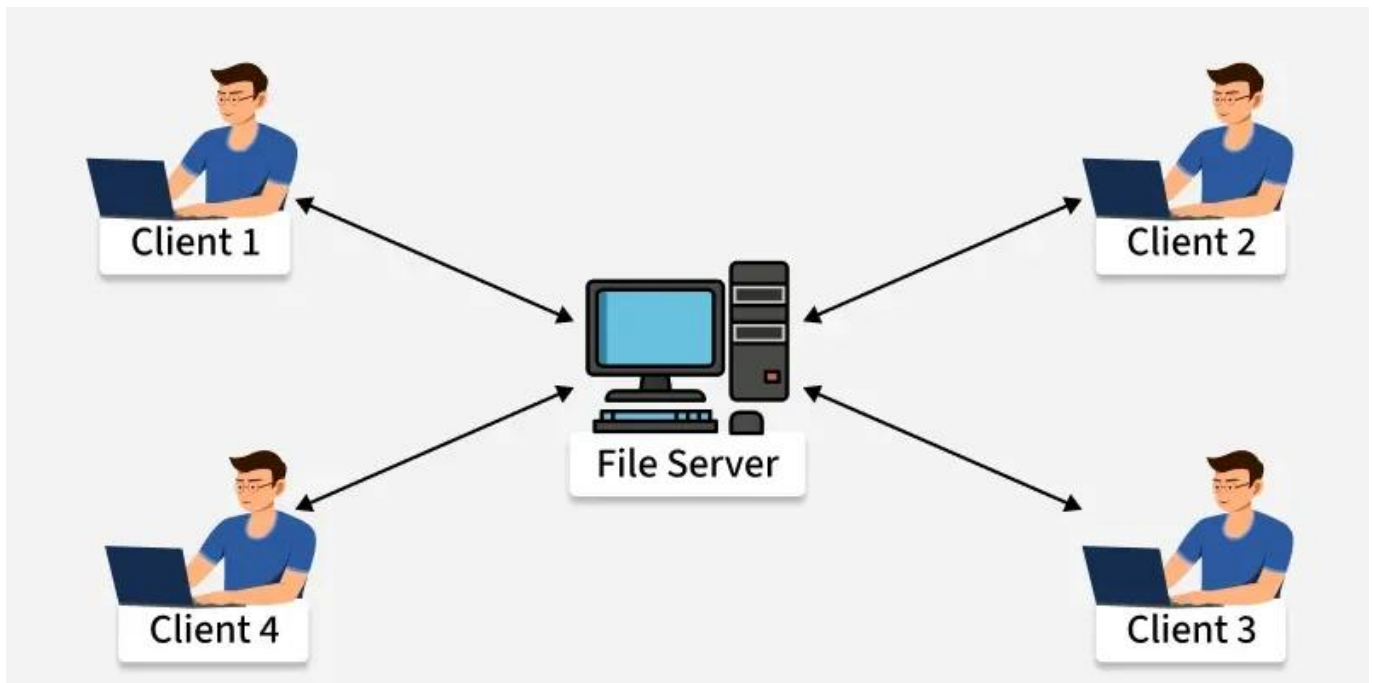
Architecture of Distributed OS



6. Network Operating System

A Network Operating System (NOS) runs on a server and manages data, users, security, applications, and other network functions. It allows shared access to files, printers, and

resources within a small private network. Users can see the configuration and connections of other users, which is why these systems are considered [tightly coupled systems](#).



7. Real-Time Operating System

These types of OSs serve real-time systems. The time interval required to process and respond to inputs is very small. This time interval is called response time. Real-time systems are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Types of Real-Time Operating Systems

- **Hard Real-Time Operating System:** Used where strict timing is essential and any delay is unacceptable, such as airbags or automatic parachutes. These systems avoid virtual memory to ensure immediate response.
- **Soft Real-Time Operating System:** Used where timing is important but minor delays are acceptable. These systems aim to give quick and predictable responses but do not require perfect accuracy. They are commonly used in multimedia applications, gaming, video streaming, and other interactive tasks.

8. Mobile Operating Systems

Mobile operating systems are designed specifically for mobile devices such as smartphones and tablets. Examples of such operating systems are Android and iOS. These operating

systems manage the hardware and software resources of the device, providing a platform for running applications and ensuring a seamless user experience

Linux

Linux is one of the most widely used open-source operating systems. It's fast, secure, stable, and powers everything from smartphones and servers to cloud platforms and IoT devices. Linux is especially popular among developers, system administrators, and DevOps professionals.

1. Understanding Linux Basics

In this section, we will cover the basics of Linux operating system, how it works, and how it's different from Unix. We'll learn about the concept of Linux distributions (distros), why they exist, and how to choose the right one based on your needs whether you're a beginner, developer, or system admin.

➤ **Operating system –**

Linux is based on the UNIX operating system. UNIX is a powerful, multi-user, multitasking operating system originally developed in the 1970s at AT&T Bell Labs. It laid the foundation for many modern operating systems, including Linux.

- **Linux is free and open-source, accessible to everyone.**
- **This promotes global collaboration and innovation.**
- **Linux offers efficient performance and strong security.**
- **It works well across many devices and industries.**

➤ **Distributions in Linux –**

A Linux distribution is an operating system built on the Linux kernel, bundled with essential software, tools, and package managers, customized to serve different users like developers, enterprises, cybersecurity professionals, and general users.

What is a Linux Distribution?

- A complete OS built on top of the Linux kernel.
- Includes system software, libraries, and applications needed to run Linux on any device.

What does a distro include?

- Linux Kernel
- System Libraries
- Essential Tools & Utilities (shell, compilers, basic apps)

Batch - B60

1. Ubuntu

- What it is: One of the most popular and beginner-friendly Linux distros.
- Why it's good: Easy to install, huge community, tons of software support.
- Key Features:
 - LTS (Long-Term Support) versions
 - Big software repository
- Target Users: Beginners, general everyday users

2. Kali Linux

- What it is: A distro built mainly for cybersecurity and ethical hacking.
- Why it's used: Comes with 600+ hacking and forensics tools pre-installed.
- Key Tools: Nmap, Metasploit, Wireshark, Burp Suite
- Key Features:
 - Rolling updates
 - Designed for security testing
- Target Users: Ethical hackers, cybersecurity students, penetration testers

3. Debian

- What it is: One of the oldest and most stable Linux distros.
- Why it's special: Base for many other distros (like Ubuntu).
- Key Features:
 - Super reliable
 - Highly secure
 - Very versatile
- Target Users: Advanced users, developers who prefer stability

4. Fedora

- What it is: A cutting-edge distro backed by Red Hat.
- Why people use it: Always ships the newest tech and updates.
- Key Features:
 - Latest software
 - Great for development
 - Strong support from Red Hat
- Target Users: Developers, tech enthusiasts

5. Arch Linux

- What it is: A minimalist distro for people who want total control.
- Why it's unique: You build everything yourself and customize from scratch.
- Key Features:
 - Rolling release

- Lightweight and highly customizable
- Target Users: Advanced users, tinkerers, Linux pros

6. CentOS

- What it is: A free, stable server distro based on RHEL.
- Why it's used: Rock-solid performance for enterprise and production servers.
- Key Features:
 - Enterprise-grade stability
 - Long-term support
- Target Users: Servers, enterprise environments, data centres

7. Linux Mint

- What it is: A super user-friendly distro that feels similar to Windows.
- Why beginners love it: Smooth experience + ready-to-use out of the box.
- Key Features:
 - Easy UI
 - Simple and clean workflow
- Target Users: Beginners, users switching from Windows

Benefits of Using Linux Distributions –

Cost-Free & Open Source

- Most distros are free — no licenses, no tension.
- Open-source → anyone can view, edit, and share the code.

Highly Customizable

- Full control over look, features, and system behavior.
- Choose your own desktop environment, file system, and tools.

Strong Security

- Naturally less prone to viruses/malware.
- Regular updates + strict permission system = solid security.

Resource Efficient

- Runs smoothly even on old or low-spec PCs.
- Lightweight distros (Lubuntu, Puppy Linux) revive old machines.

Developer-Friendly

- Perfect for coding — supports almost every language + dev tool.
- Many distros ship with compilers, editors, and debugging tools.

Huge Variety (Choose Your Vibe)

- Distro for every need:
 - Ubuntu → general use
 - CentOS → servers
 - Kali Linux → security/hacking
- You pick what suits your workflow.

Stability & Reliability

- Extremely stable, especially for servers and critical systems.
- Consistent performance over long periods.

Big Community Support

- Massive global community → forums, guides, wikis everywhere.
- Easy troubleshooting + endless learning resources.

Key Components of a Linux Distribution –

Linux Kernel

- The heart of every Linux distro.
- Manages hardware, memory, CPU processes, and device drivers.
- Acts as a bridge between apps and hardware.

System Libraries & Utilities

- Core libraries (like glibc) provide standard functions for applications.
- Apps use these libraries to interact with the kernel via system calls.
- Essential for running programs smoothly.

User Interface (UI)

Linux gives two ways to interact with the system:

GUI (Graphical User Interface)

- User-friendly, visual desktops.
- Popular GUIs: GNOME, KDE, XFCE.

CLI (Command Line Interface)

- Terminal-based control.
- Preferred by advanced users for speed + flexibility.

Software Packages

Distros come with ready-to-use software like:

- LibreOffice → productivity

- Firefox/Chromium → web browsing
- VLC → media
- Compilers/IDEs → development

These tools vary depending on the distro's purpose.

Package Management System

Handles installing, updating & removing software.

- APT → Debian/Ubuntu
- YUM/DNF → Red Hat, Fedora, CentOS
- Pacman → Arch Linux

Makes software management fast and organized.

Configuration & Customization

- Change system settings: network, users, security, display, etc.
- Linux is fully customizable → perfect for personal, developer, or enterprise use.

Support & Community

- Strong documentation, wikis, and active forums.
- Community-driven help, bug fixes, and updates.
- Makes learning + troubleshooting way easier.

Use Case–Based Classification of Linux Distro

◇ Enterprise / Business

- Red Hat, CentOS, Rocky Linux
- Known for stability, long-term support, and security.

◇ Cybersecurity & Pen Testing

- Kali Linux, Parrot OS
- Preloaded with hacking + forensics tools.

◇ Developers & Programmers

- Fedora, Arch Linux, Ubuntu
- Latest packages, dev tools, and flexibility.

◇ Multimedia Production

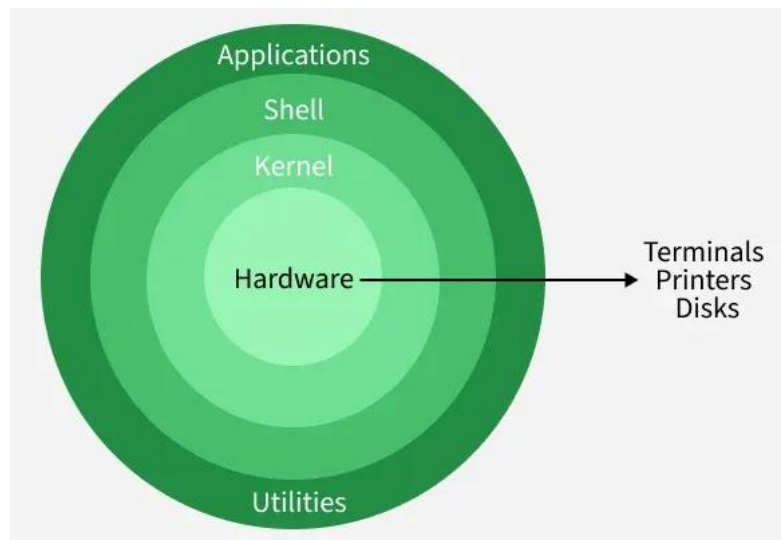
- Ubuntu Studio, AV Linux
- Optimized for audio/video editing and creative workflows.

◇ Lightweight / Old Hardware

- Lubuntu, Puppy Linux
- Extremely fast, low resource usage, revives old PCs.

Architecture of Linux –

Linux architecture refers to the layered structure of the Linux operating system that defines how its components - such as the kernel, shell, system libraries, and hardware - interact with each other to manage system resources and execute user programs efficiently. It has the following components:



1. Kernel

The kernel is the core of the Linux operating system that manages hardware resources and controls communication between software and hardware.

- Handles process management, memory, and device control.
- Prevents conflicts between multiple running programs.

Types of Kernels: Monolithic, Microkernel, Hybrid, Exokernel

2. System Libraries

System libraries provide essential functions that allow applications to interact with the kernel without needing to access it directly.

- Contain reusable pre-written code for common system operations.
- Act as an interface between applications and the Linux kernel.

3. Shell

The shell is the command-line interface that allows users to communicate with the operating system by entering commands.

- Interprets and executes user commands.
- Acts as a bridge between user actions and kernel processing.

4. Hardware Layer

The hardware layer consists of physical components that execute commands and provide system resources.

- Includes CPU, RAM, storage, and input/output devices.
- Communicates with the OS using device drivers and kernel services.

5. System Utilities

System utilities are built-in tools that help users manage, configure, and maintain the operating system.

- Used for tasks like software installation, user management, and monitoring.
- Simplify system administration processes for both beginners and administrators.

Applications of Linux Operating System –



Linux Commands –

Linux commands are essential tools used in the Command Line Interface (CLI) to interact with the operating system. They enable users to navigate directories, manage files, monitor system performance, and troubleshoot network issues.

File and Directory Commands

ls

Lists the contents of the current directory.

cd

Changes the current working directory.

pwd

Displays the full path of the current directory.

cp, mv, rm

- **cp**: Copies files or directories
- **mv**: Moves or renames files or directories
- **rm**: Deletes files or directories

System Monitoring Commands

free

Shows memory usage, including RAM and swap.

df

Displays disk space usage of file systems.

top

Shows real-time information about running processes and system resource usage.

Network Commands

netstat

Displays active network connections, routing tables, and ports in use.

ifconfig

Shows or configures network interface information.

ping

Checks network connectivity to a specific host or server

Sr.no	Linux	Windows
1.	Linux is an open-source operating system.	Windows is not an open-source operating system.
2.	Linux is free of cost .	Windows is paid and requires a license.
3.	File names are case-sensitive , meaning <code>file.txt</code> and <code>File.txt</code> are different.	File names are case-insensitive , meaning <code>file.txt</code> and <code>File.txt</code> are treated the same.
4.	Uses a monolithic kernel .	Uses a hybrid kernel .
5.	More efficient and stable , especially for servers and developers.	Less efficient due to resource-intensive processes.

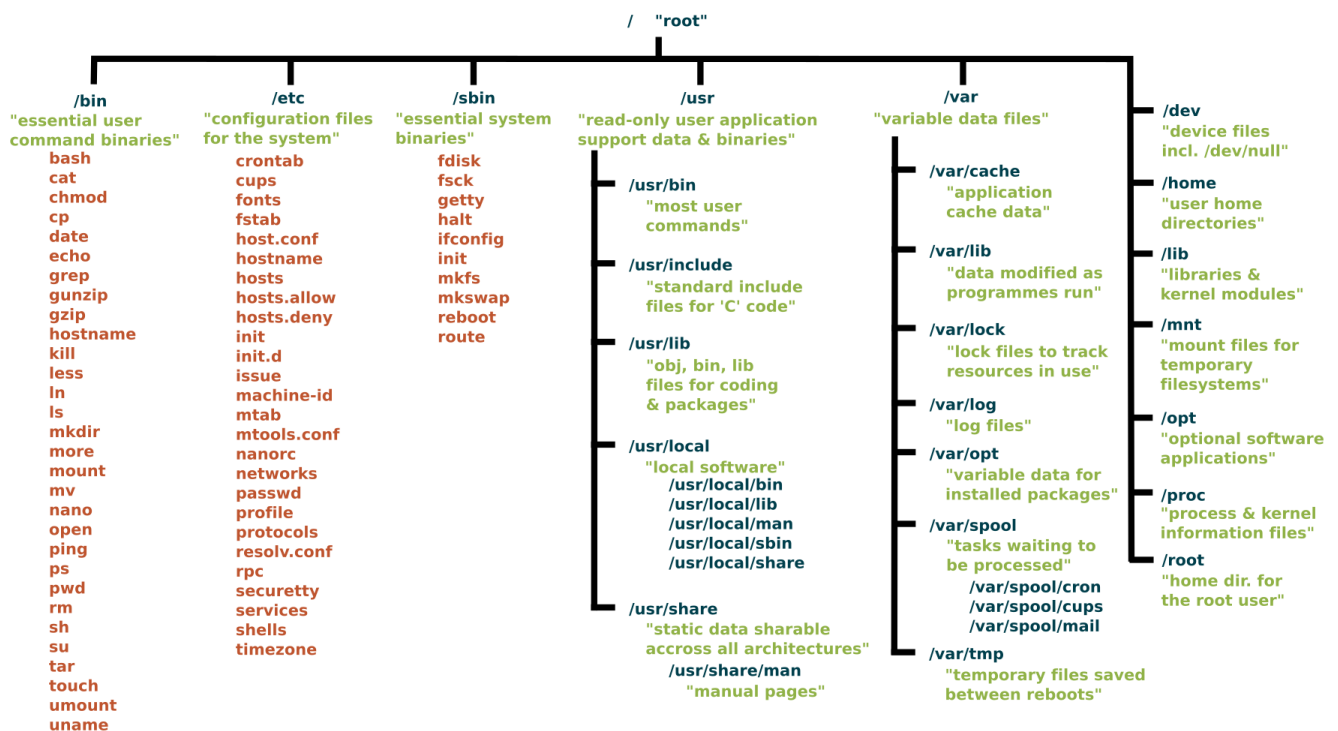
6.	Uses forward slash (/) for directory separation.	Uses backslash (\) for directory separation.
7.	More secure with better user control and fewer vulnerabilities.	Less secure due to higher susceptibility to malware and viruses.
8.	Preferred by hackers and security experts due to its open-source nature and control.	Not widely used for hacking as it lacks built-in security tools.
9.	Has 3 types of user accounts : (1) Regular, (2) Root, (3) Service Account.	Has 4 types of user accounts : (1) Administrator, (2) Standard, (3) Child, (4) Guest.
10.	Root user has all administrative privileges.	Administrator user has all administrative privileges.
11.	In Linux, you can have two files with the same name but different cases (File.txt and file.txt).	In Windows, you cannot have two files with the same name in the same folder.

Directory Structure in Linux –

Linux follows a hierarchical directory structure, starting from the root directory /.

Every file and directory in a Linux system originates from /, making it the top-most level of the filesystem.

Below is a clear explanation of the standard Linux directory structure and its purpose.



1. / (Root Directory)

The top-level directory of the Linux filesystem.
All other directories and files branch out from here.

2. /bin (User Binaries)

Contains essential command binaries used by all users.
Examples: ls, cp, mv, cat, grep
These commands are needed even when no other filesystems are mounted.

3. /sbin (System Binaries)

Stores system administration commands.
Used mostly by the root user.
Examples: reboot, ifconfig, fsck

4. /etc (Configuration Files)

Holds system-wide configuration files.
Examples:

- /etc/passwd
- /etc/ssh/sshd_config
- /etc/fstab

No binaries are stored here

5. /home (User Home Directories)

Each user gets a personal directory under /home.

Examples:

/home/sahil

/home/student

Stores user-specific files, documents, configs.

6. /root (Root User Home Directory)

Home directory for the root (administrator) user.

Not the same as /.

7. /boot (Boot Files)

Contains all files required to boot the system.

Examples:

- Kernel files
- GRUB bootloader files

Without this directory, the system cannot start.

8. /dev (Device Files)

Contains device files representing hardware.

Examples:

- /dev/sda (hard disk)
- /dev/tty (terminal)
- /dev/null

Linux treats devices as files.

9. /proc (Process Information)

A virtual filesystem providing process and system information.

Examples:

- /proc/cpuinfo
- /proc/meminfo
- /proc/[PID]

This directory is dynamically generated.

10. /sys (System Information)

Another virtual filesystem providing device and kernel information.

Used for system management and tuning.

11. /var (Variable Files)

Stores files that change frequently.

Examples:

- Logs → /var/log

- Mail → /var/mail
- Cache files
- Temporary databases

12. /tmp (Temporary Files)

Stores temporary files created by apps or users.
Files here are usually deleted on reboot.

13. /usr (User Programs)

One of the largest directories. Contains user-level applications and data.
Important subdirectories:

- /usr/bin → non-essential user binaries
- /usr/sbin → non-essential system binaries
- /usr/lib → libraries
- /usr/share → shared files

14. /opt (Optional Software)

Used for installing third-party software not part of the OS.

Example:

Applications like Chrome or VMware may install here.

15. /lib (Libraries)

Stores essential shared libraries needed by binaries in /bin and /sbin.
Includes kernel modules: /lib/modules

16. /media (Removable Media)

Mount point for external drives.

Examples:

- USB drives
- CDs
- External HDDs

17. /mnt (Temporary Mount Point)

Used by admins to temporarily mount filesystems.

Types of Shell in Linux (Short + Detailed Notes)

A *shell* is a command interpreter that allows users to interact with the OS.
Different shells offer different features, scripting capabilities, and user experience.

1. Bourne Shell (sh)

- The original Unix shell by Stephen Bourne.

- Very stable and widely supported.
- Limited features compared to modern shells.
- Script compatibility is very high.

Check version:

sh --version

2. Bash (Bourne Again Shell) – Most Common

- Default shell on most Linux systems.
- Supports command history, aliases, functions, scripting.
- Very user-friendly; best for beginners & scripting.

Check version:

bash --version

3. Z Shell (zsh)

- More advanced and customizable.
- Supports themes and plugins (Oh-My-Zsh).
- Auto-completion, correction, suggestions.
- Popular among developers.

Switch to zsh:

zsh

4. Korn Shell (ksh)

- Developed AT&T Bell Labs.
- Features from both Bourne & C Shell.
- Strong scripting capabilities.
- Often used in enterprise systems.

Start:

ksh

5. C Shell (csh)

- Syntax similar to C programming language.
- Provides job control & aliasing.
- Not very popular for scripting today due to inconsistencies.

Start:

csh

6. Tcsh (Enhanced C Shell)

- Improved version of csh.
- Includes command-line editing, auto-complete.
- More stable than csh.

Start:
tcsh

7. Fish Shell (Friendly Interactive Shell)

- Very modern and user-friendly.
- Real-time syntax highlighting & suggestions.
- Not fully POSIX-compliant (scripts not portable).

Start:
fish

8. Viewing Available Shells

cat /etc/shells

9. Change Default Shell

chsh -s /bin/bash

chsh -s /bin/zsh

10. Summary Table

Shell	Full Name	Famous For
sh	Bourne Shell	Original shell, high compatibility
bash	Bourne Again Shell	Most common, powerful scripting
zsh	Z Shell	Modern, customizable, developer-friendly
ksh	Korn Shell	Enterprise scripting
csh	C Shell	C-like syntax
tcsh	Tenex C Shell	Better csh with editing features
fish	Friendly Interactive Shell	Very user-friendly UI, auto-suggestions

Linux Commands Categorized

1. File and Directory Management

- **ls** – List directory contents
- **cd** – Change directory
- **pwd** – Show current directory
- **cp** – Copy files/directories
- **mv** – Move or rename files/directories
- **rm** – Remove files/directories
- **mkdir** – Create a new directory
- **rmdir** – Remove an empty directory
- **touch** – Create an empty file or update timestamp
- **cat** – Display file contents

- **nl** – Show file contents with line numbers
- **head** – Show beginning of a file
- **tail** – Show end of a file
- **chmod** – Change file permissions
- **chown** – Change file owner
- **chgrp** – Change group ownership
- **file** – Determine file type
- **find** – Search for files/directories
- **locate** – Quickly find files using database
- **stat** – Display detailed file information

2. System Information Commands

- **uname** – Show system/kernel information
- **hostname** – Display or set system hostname
- **top** – Real-time process monitoring
- **htop** – Improved version of top (if installed)
- **free** – Show memory usage
- **df** – Show disk space usage
- **du** – Show directory/file size
- **uptime** – Show system uptime and load
- **vmstat** – Show memory, process, CPU statistics
- **dmesg** – Kernel messages
- **lsblk** – List block devices
- **lscpu** – CPU information
- **lsusb** – USB device information
- **lspci** – PCI device information

3. Process Management

- **ps** – List processes
- **top** – Monitor processes
- **kill** – Terminate processes
- **killall** – Kill processes by name
- **nice** – Start a process with priority
- **renice** – Change running process priority
- **jobs** – Show background jobs
- **bg** – Move job to background
- **fg** – Move job to foreground

4. User and Group Management

- **useradd** – Add new user
- **userdel** – Delete user
- **usermod** – Modify user details
- **passwd** – Change password
- **groupadd** – Add new group
- **groupdel** – Delete group
- **groups** – Show user groups
- **id** – Show user identity

5. File Compression and Archiving

- **tar** – Archive files
- **gzip** – Compress files
- **gunzip** – Decompress gzip files
- **zip** – Create zip archives
- **unzip** – Extract zip archives

6. Networking Commands

- **ping** – Test connectivity
- **ifconfig** – Configure network interfaces
- **ip** – Advanced network and routing tool
- **netstat** – Network connections, routing, ports
- **ss** – Replacement for netstat
- **nmcli** – Network Manager CLI
- **curl** – Transfer data from/to a server
- **wget** – Download files from the web
- **scp** – Secure copy (remote transfer)
- **ssh** – Remote login
- **traceroute** – Trace network path
- **dig** – DNS lookup
- **host** – DNS queries

7. Package Management

Debian/Ubuntu (APT)

- **apt update** – Update package list
- **apt upgrade** – Upgrade packages
- **apt install** – Install package
- **apt remove** – Remove package
- **apt purge** – Remove package with config files
- **apt search** – Search for a package

RHEL/Fedora/CentOS (YUM/DNF)

- **yum install / dnf install**
- **yum update / dnf update**
- **yum remove / dnf remove**

Arch Linux

- **pacman -S** – Install
- **pacman -R** – Remove
- **pacman -Syu** – Update system

8. Disk and Filesystem Commands

- **fdisk** – Disk partitioning tool
- **mkfs** – Create filesystem
- **mount** – Mount a filesystem
- **umount** – Unmount filesystem
- **fsck** – File system check
- **blkid** – Identify devices
- **parted** – Disk partition tool
- **resize2fs** – Resize ext2/ext3/ext4 filesystem

9. Permissions and Ownership

- **chmod** – Change permissions
- **chown** – Change file owner
- **chgrp** – Change group
- **umask** – Default permission settings

10. System Services

- **systemctl** – Manage **systemd** services
- **service** – **Start/stop services** (older systems)
- **journalctl** – View system logs

11. Text Processing Commands

- **grep** – Search text
- **awk** – Powerful text processing
- **sed** – Stream editor for modifying text
- **sort** – Sort lines
- **cut** – Cut portions of text
- **wc** – Word/line count
- **uniq** – Remove duplicates
- **tr** – Translate/replace characters

12. Shell & Scripting Commands

- **echo** – Print text
- **read** – Read input
- **sh / bash** – Shell execution
- **source** – Run script in current shell
- **alias** – Create command shortcuts

13. System Administration

- **shutdown** – Power off
- **reboot** – Restart system
- **crontab** – Schedule tasks
- **sudo** – Run commands as root
- **su** – Switch users
- **export** – Set environment variables

14. Log Management

- **cat /var/log/syslog** – System logs
- **tail -f /var/log/messages** – Live log monitoring
- **journalctl** – View systemd logs

Summary of **ls** Options

Option	Meaning	Why It Is Used
-l	Long listing	Check permissions, size, timestamp
-a	Show hidden files	View config & system files
-h	Human-readable sizes	Easier file size reading
-R	Recursive listing	View directory tree
-S	Sort by size	Find large files
-t	Sort by time	Track recent changes
-F	Show file types	Identify dirs, executables
-d */	Show only directories	Quick folder overview

The **cd** Command (Change Directory)

Command	Meaning	Use Case
cd	Go to home directory	Quick return to workspace
cd ~	Home directory	Works in scripts
cd .	Stay where you are	Script logic

cd ..	Move up one level	Navigate backwards
cd -	Previous directory	Switching paths
cd /path	Absolute navigation	System directory access
cd folder	Relative navigation	Quick moves
cd ../..	Two levels up	Escaping deep folders

The **pwd** Command (Print Working Directory)

1. What is **pwd**?

pwd stands for Print Working Directory.

It displays the full absolute path of the directory you are currently working in.

It is a simple but essential command to know *where you are* in the Linux filesystem at any moment.

Notes About **pwd**

- It **always prints absolute path**, never relative
- It is a built-in shell command (not an external binary)
- Safe to run anytime, doesn't modify anything
- Very important for beginners to confirm their path

Command: **cp** (Copy Files and Directories)

The cp command is used to copy files or directories from one location to another in Linux.

Purpose	Explanation
Create duplicates of files	Copy a file to another name or location
Backup important files	Make safe copies before editing or deleting
Copy whole folders	Useful in projects, configs, and data migration
Copy with attributes	Preserve permissions, timestamps, ownership

Syntax

cp [options] source destination

- **source** = file or directory to copy
- **destination** = where to copy it

Important cp Options

Option	Meaning	Why It Is Used
cp file1 file2	Copy a file	Basic copying

cp source dest/	Copy file into directory	Organizing files
cp -r	Copy directories recursively	Required for folders (else cp fails)
cp -v	Verbose mode	Shows what is being copied
cp -i	Interactive mode	Prevents overwriting without confirmation
cp -f	Force overwrite	Useful in scripts or automation
cp -p	Preserve file attributes	Keeps timestamps, permissions, ownership
cp -u	Copy only when source is newer	Used for backups or syncing

Example	Explanation
cp file.txt backup.txt	Copies file.txt to backup.txt
cp file.txt /home/sahil/	Copies file into a directory
cp -r project/ /home/sahil/	Copies a full folder recursively
cp -v file.txt /tmp/	Shows copying progress
cp -i notes.txt notes.txt	Prompts before overwriting
cp -p report.pdf /backup/	Copies while preserving permissions & timestamps
cp -u *.txt /archive/	Copies only updated text files

Command: **mv** (Move or Rename Files and Directories)

The mv command is used to **move** files/directories from one place to another or **rename** them.

It does not create a copy; it *relocates* or *renames* the original file.

Why mv is Used

Purpose	Explanation
Move files	Shift files between directories
Rename files	Change file or folder names
Organize directories	Move projects, logs, documents
Replace old files	Move new files to overwrite old ones
Manage storage	Move data to other disks or partitions

Syntax

mv [options] source destination

- **source** = file/directory to move or rename
- **destination** = new path or new filename

Important mv Options

Option	Meaning	Why It Is Used
mv file1 file2	Renames file1 to file2	Simple renaming
mv file /folder/	Moves file into folder	Organizing files
mv dir1 /path/	Moves a directory	Restructuring projects
mv -i	Interactive mode	Asks before overwriting
mv -f	Force move	Overwrites without asking
mv -v	Verbose mode	Shows exactly what is moved
mv -n	No overwrite	Prevents overwriting existing files

Detailed Examples

Example	Explanation
mv oldname.txt newname.txt	Renaming a file
mv report.pdf /home/sahil/Documents/	Moving file to directory
mv project/ /home/sahil/	Moving a folder
mv -i data.csv backup.csv	Asks before overwriting backup.csv
mv -f new.log /var/log/	Force overwrites if file exists
mv -v *.txt /archive/	Moves all text files with progress shown
mv -n file.txt copy.txt	Does nothing if copy.txt exists

Command: rm (Remove Files and Directories)

rm is used to delete files and directories permanently from the filesystem. Files removed with rm do *not* go to any recycle bin — deletion is immediate. Because of this, rm must be **used carefully**.

Why rm is Used

Purpose	Explanation
Delete files	Remove unnecessary data
Delete directories	Clean up folders (with -r)
Remove large sets of files	Automate cleanup in scripts
Force delete protected files	Using -f when permissions block removal

Syntax

```
rm [options] file_or_directory
```

Important rm Options (Meaning + Why Used)

Option	Meaning	Why It Is Used
rm file	Delete a file	Basic removal
rm -r directory	Recursive delete	Required to delete folders
rm -f file	Force delete	Skip confirmation, ignore errors
rm -rf directory	Recursive + force	Delete folder with all contents
rm -i file	Interactive mode	Asks before deleting
rm -v file	Verbose mode	Shows what is being deleted
rm *.txt	Delete file patterns	Useful for batch removal

Detailed Examples

Example	Explanation
rm file.txt	Deletes file.txt
rm -i file.txt	Confirms before deletion
rm -r folder/	Deletes folder and its files
rm -rf /tmp/work/	Deletes folder forcefully without asking
rm -v notes.txt	Shows deletion action
rm *.log	Deletes all .log files in directory
rm -r Projects/*	Deletes all content inside Projects folder

Common Mistakes and Warnings

Issue	Explanation
rm -rf /	DANGEROUS — deletes entire system
rm -rf *	Deletes everything in current directory
rm file cannot delete directories	Must use -r for folders
No undo after deletion	Files are permanently removed

Using sudo rm -rf without care	Can break the whole OS
--------------------------------	------------------------

Safety Tips

Tip	Why
Use rm -i if unsure	Prevents accidental deletion
Check your path using pwd	Avoid deleting wrong directory
Prefer rm -r folder/ instead of rm -rf	Less risk
Avoid running rm as root unless necessary	Root deletion is permanent

Command: **mkdir** (Make Directory)

`mkdir` is used to **create new directories (folders)** in Linux.

Why **mkdir** is Used

Purpose	Explanation
Create folders	Build directory structures for projects and files
Create nested directories	Using <code>-p</code> to make multiple levels at once
Organize files	Set up workspace or storage locations

Syntax

```
mkdir [options] directory_name
```

Important Options for `mkdir`

Option	Meaning	Why Used
<code>mkdir dir</code>	Creates a directory	Basic usage
<code>mkdir -p path/sub1/sub2</code>	Creates nested directories	Avoids errors if parent dirs don't exist
<code>mkdir -v dir</code>	Verbose mode	Shows confirmation of created folders

Examples

Command	Explanation
<code>mkdir project</code>	Creates a directory named project
<code>mkdir -p projects/2025/devops</code>	Creates full nested path
<code>mkdir -v logs</code>	Shows message while creating folder

Summary Table

Command	Meaning
<code>mkdir</code>	Create directory
<code>mkdir -p</code>	Create nested directories
<code>mkdir -v</code>	Show creation details

Command: **rmkdir** (Remove Directory)

`rmkdir` removes **empty directories only**.

Why **rmkdir** is Used

Purpose	Explanation
Clean up empty directories	Safe deletion (only works if folder is empty)
Administrative cleanup	Remove unnecessary directory structure

Syntax

```
rmkdir [options] directory
```

Important Options

Option	Meaning	Why Used
<code>rmkdir dir</code>	Removes empty directory	Basic usage
<code>rmkdir -p path/sub1/sub2</code>	Removes multiple empty directories in path	Useful for nested structure cleanup

Examples

Command	Explanation
<code>rmdir oldfolder</code>	Deletes folder if empty
<code>rmdir -p test/project/demo</code>	Deletes demo → project → test if all are empty

Summary Table

Command	Meaning
<code>rmdir</code>	Remove empty directory
<code>rmdir -p</code>	Remove nested empty directories

Command: touch (Create or Update File)

`touch` creates an empty file or updates its timestamp.

Why touch is Used

Purpose	Explanation
Create empty files	Needed in scripting, config, placeholders
Update timestamps	Useful for build tools and automation
Test file permissions	Quickly create files in directories

Syntax

```
touch filename
```

Examples

Command	Explanation
<code>touch notes.txt</code>	Creates an empty file
<code>touch file1 file2</code>	Creates multiple files
<code>touch -a file.txt</code>	Updates access time
<code>touch -m file.txt</code>	Updates modify time

Summary Table

Command	Meaning
touch file	Create empty file
touch -a	Update access time
touch -m	Update modification time

4. Command: **cat** (Display or Combine Files)

cat is used to **view, create, or concatenate** files.

Why **cat** is Used

Purpose	Explanation
Read file content	Quick viewing
Create files	Using redirection
Merge files	Combine multiple files

Syntax

```
cat [options] file
```

Important Options

Option	Meaning	Why Used
cat file	Display file	Simple viewing
cat > file	Create/overwrite file	Manual input
cat file1 file2 > newfile	Combine files	Useful for logs, text files
cat -n file	Number lines	Code or log referencing

Examples

Command	Explanation
cat notes.txt	View file

cat > test.txt	Create file
cat file1 file2 > merged.txt	Combine files
cat -n script.sh	Show line numbers

Summary Table

Command	Meaning
cat	Display content
cat > file	Create file
cat file1 file2	Merge files
cat -n	Number lines

Command: **find** (Search Files and Directories)

`find` searches files in directory hierarchies based on name, size, type, modification date, etc.

Why **find** is Used

Purpose	Explanation
Search files anywhere	Even deep directory structures
Find by name, type, size	Very flexible search
Execute actions on found files	Delete, move, print

Syntax

```
find [path] [criteria] [action]
```

Important Options & Filters

Option	Meaning	Why Used
-name	Search by name	Most common search
-type	Search by file type	f = file, d = directory
-size	Search by size	Find big or small files

-mtime	Search by modification time	Logs, backups
-delete	Delete matched files	Automation
-exec	Run command on results	Move, copy, delete

Examples

Command	Explanation
<code>find /home -name "*.txt"</code>	Find all text files
<code>find . -type d</code>	Find all directories
<code>find /var -size +100M</code>	Files larger than 100 MB
<code>find . -mtime -1</code>	Modified in last 1 day
<code>find . -name "*.log" -delete</code>	Delete all log files
<code>find . -name "*.pdf" -exec cp {} /backup \;</code>	Copy found PDFs to backup

Summary Table

Command	Meaning
<code>find -name</code>	Find by name
<code>find -type</code>	Find by type
<code>find -size</code>	Find by size
<code>find -delete</code>	Delete files
<code>find -exec</code>	Execute command on results

6. Command: **grep** (Search Text in Files)

`grep` searches for patterns or text inside files.

Why **grep** is Used

Purpose	Explanation
---------	-------------

Search logs	Find errors or messages
Filter output	Extract specific data
Pattern matching	Text analysis, debugging
Combine with other commands	Pipes (

Syntax

grep [options] pattern file

Important Options

Option	Meaning	Why Used
grep text file	Search for text	Basic usage
grep -i	Ignore case	Case-insensitive search
grep -r	Recursive search	Search in folders
grep -n	Show line numbers	Useful for debugging
grep -v	Invert match	Show lines NOT matching
grep -E	Extended patterns	Complex searches

Examples

Command	Explanation
grep "error" logfile.txt	Find "error" in logs
grep -i "failed" auth.log	Case-insensitive search
grep -n "import" script.py	Shows line numbers
grep -r "password" /etc	Search inside directories
grep -v "INFO" logfile.txt	Show lines without INFO

Summary Table

Command	Meaning
grep text file	Basic search
grep -i	Case-insensitive
grep -r	Recursive search
grep -n	Show line numbers
grep -v	Inverted search

7. Command: **chmod** (Change File Permissions)

`chmod` modifies file and directory permissions.

Why **chmod** is Used

Purpose	Explanation
Control access	Restrict or allow reading, writing, executing
Set permissions for scripts	Make scripts executable
Secure files	Remove unwanted access

Syntax

`chmod [options] mode file`

Modes can be **symbolic** (u,g,o) or **numeric** (755).

Important Modes

Mode	Meaning
755	Owner full, others read+execute
644	Owner read+write, others read
700	Owner only, fully private

Examples

Command	Explanation
<code>chmod 755 script.sh</code>	Makes script runnable
<code>chmod 644 notes.txt</code>	Standard read/write file
<code>chmod u+x file.sh</code>	Add execute permission to owner
<code>chmod g-w file.txt</code>	Remove write permission from group

Summary Table

Command	Meaning
chmod 755	Make executable
chmod 644	Standard file permissions
chmod u+x	Add execute
chmod g-w	Remove write

8. Command: **chown** (Change Ownership)

`chown` changes the **owner** and/or **group** of files and directories.

Why **chown** is Used

Purpose	Explanation
Assign ownership	When creating or moving files
Fix file permissions	After downloads or installations
Manage multi-user systems	Give correct access

Syntax

`chown [options] owner:group file`

Important Options

Option	Meaning
<code>chown user file</code>	Change owner
<code>chown user:group file</code>	Change owner and group
<code>chown -R user:group dir</code>	Recursive change

Examples

Command	Explanation
<code>chown sahil file.txt</code>	Make sahil the owner
<code>chown root:root config</code>	Change owner and group
<code>chown -R user:dev team/</code>	Recursively assign ownership

Summary Table

Command	Meaning
chown user file	Change owner
chown user:group file	Change owner and group
chown -R	Recursive ownership change

Command: **ps** (Process Status)

1. What is **ps**?

ps is used to display information about running processes on a Linux system. It shows process IDs, memory usage, CPU usage, and who owns the process. It is one of the most important commands for system monitoring and troubleshooting.

2. Why **ps** is Used

Purpose	Explanation
Check running processes	See what is currently running on the system
Find a process ID (PID)	Needed for killing or monitoring processes
Monitor resource usage	CPU, memory details of active processes
Debugging programs	Check if a script or service is running
Security checks	Identify unknown or suspicious processes

3. Syntax

ps [options]

4. Important **ps** Options (Meaning + Why Used)

Option	Meaning	Why Used
ps	Shows processes of current terminal	Basic view for local shell
ps -e	Show all processes	System-wide process view
ps -f	Full-format listing	More detailed info
ps -u username	User-specific processes	Check what a user is running
ps aux	Show all processes in full format	Most common for admins
ps -ef	System-wide detailed listing	Used for debugging services
ps -eH	Show process hierarchy	View parent-child relations
ps -eo pid,cmd	Custom output	Useful for scripting

5. Detailed Examples

Command	Explanation
ps	Shows processes in current shell
ps -e	Shows all system processes
ps -f	Full details: PPID, UID, time
ps -u sahil	Processes owned by sahil
ps aux	Most detailed overview of all processes
`ps aux	grep nginx`
ps -eo pid,ppid,cmd	Show only PID, PPID, and command
`ps -ef	grep sshd`

6. Fields Displayed by **ps**

Field	Meaning
PID	Process ID
PPID	Parent process ID
UID	User who started the process
CMD	Command that launched the process
TIME	CPU time used
STAT	Process state (R, S, Z, T)

7. Summary Table

Command	Meaning	Use Case
ps	Current shell processes	Simple check
ps -e	All system processes	System-wide view
ps aux	All processes with details	Admin troubleshooting
ps -f	Full detailed output	See parent-child relation
ps -u user	User-specific processes	Multi-user systems
`ps -ef	grep name`	Search process
ps -eo fields	Custom output	Scripting

Command: **top** (Real-Time Process Monitoring)

1. What is **top**?

top is a real-time process monitoring tool in Linux.

It displays system performance details such as:

- CPU usage
- Memory usage
- Running processes

- Load average
- Process IDs
- System uptime

It updates dynamically and is one of the most important tools for system administrators.

2. Why **top** is Used

Purpose	Explanation
Real-time monitoring	View live CPU, RAM, and process activity
Identify high-usage processes	Find processes consuming high CPU or memory
Troubleshooting system slowdowns	Check load average and bottlenecks
Kill or control processes	Manage runaway processes
Track resource usage over time	Continuous system watching

3. Syntax

`top`

`top` runs until you exit it using:

`q`

4. Important Interactive Commands Inside **top**

Key	Meaning	Why Used
q	Quit	To exit <code>top</code>
k	Kill a process	Quickly stop high-usage processes
P	Sort by CPU usage	Find CPU-heavy tasks
M	Sort by memory usage	Find RAM-heavy tasks
N	Sort by PID	View processes by ID
H	Show threads	Debug multi-threaded apps
1	Show all CPU cores	Detailed CPU load
Shift + >	Move sort column right	Custom sorting
Shift + <	Move sort column left	Custom sorting

These keys make `top` extremely powerful and interactive.

5. Important Fields Displayed in **top**

Field	Meaning
PID	Process ID
USER	Owner of process
PR	Priority

NI	Nice value
VIRT	Virtual memory used
RES	Physical memory used
SHR	Shared memory
S	Process state (R, S, Z, T)
%CPU	CPU usage
%MEM	Memory usage
TIME+	Total CPU time
COMMAND	Name of the process

6. Examples

Example 1: Start **top**

top

Example 2: Sort processes by memory usage

Inside top:

M

Example 3: Sort processes by CPU usage

Inside top:

P

Example 4: Kill a process

Inside top:

k

Then enter PID (example):

1234

Example 5: Show all CPU cores

Inside top:

1

7. Why **top** is better than **ps** in some cases

ps	top
Shows a snapshot	Real-time updates
Not interactive	Fully interactive
Good for scripting	Good for monitoring
No sorting inside command	Sorting available anytime

8. Summary Table

Command / Key	Meaning	Use Case
top	Start real-time monitoring	System performance
q	Quit	Exit top
P	Sort by CPU	Find CPU-heavy processes
M	Sort by memory	Find memory-heavy processes
1	Show all CPU cores	Multi-core analysis
k	Kill process	Stop unresponsive tasks
H	Show threads	Debugging

1. Command: df (Disk Filesystem Usage)

df displays available and used disk space on mounted filesystems.

Why df is Used

Purpose	Explanation
Check disk space	Find free and used storage on partitions
Monitor server storage	Prevent system from filling up
Identify full partitions	Troubleshooting storage issues
Human-readable output	Using -h option for clarity

Syntax

df [options] [path]

Important Options

Option	Meaning	Why Used
df	Show disk usage	Basic usage
df -h	Human-readable	Shows in MB/GB
df -T	Show filesystem type	Identify ext4, xfs, etc.
df -i	Show inode usage	Avoid "no space left" errors due to inodes

Examples

Command	Explanation
df -h	Display disk usage in human readable form
df -i	Shows inode usage
df -T	Shows filesystem type

Summary Table

Command	Meaning
df	Disk usage
df -h	Human-readable
df -T	Show filesystem type
df -i	Show inode usage

2. Command: **du** (Directory/File Size Usage)

du reports **disk usage** of files and directories.

Why **du** is Used

Purpose	Explanation
Find folder size	Useful when cleaning storage
Find large files	Track down big directories
Human-readable format	With -h option
Summarize sizes	For quick analysis

Syntax

du [options] [path]

Important Options

Option	Meaning	Why Used
du	Show directory size	Basic usage
du -h	Human-readable	Shows size in MB/GB
du -sh folder	Summary only	Quick size overview
du -a	Show files and folders	Detailed breakdown

Examples

Command	Explanation
du -h	Show folder sizes in readable format
du -sh /var/log	Show total size of /var/log
du -ah	Show size of every file

Summary Table

Command	Meaning
du	Show disk usage
du -h	Human-readable
du -sh	Summary only
du -a	Include files

3. Command: **free** (Memory Usage)

free displays RAM and swap memory usage.

Why **free** is Used

Purpose	Explanation
Check RAM usage	Monitor system performance
Check swap usage	Identify memory pressure
Human-readable output	Using -h
Troubleshooting slow systems	Detect memory shortages

Syntax

```
free [options]
```

Important Options

Option	Meaning	Why Used
free	Basic memory info	Simple usage
free -h	Human-readable	Shows MB/GB
free -m	Show memory in MB	Common on servers

Examples

Command	Explanation
free -h	Shows RAM usage clearly
free -m	Show memory in MB

Summary Table

Command	Meaning
free	Memory usage
free -h	Human-readable
free -m	Show in MB

4. Command: **uptime** (System Uptime & Load)

uptime shows how long the system has been running, number of users, and **load average**.

Why **uptime** is Used

Purpose	Explanation
Check system longevity	How long system is running
Check load average	Useful for performance monitoring
Quick server health check	Lightweight and fast

Syntax

uptime

Example Output

14:20:10 up 5 days, 3 users, load average: 0.12, 0.15, 0.20

Summary Table

Field	Meaning
Up 5 days	System uptime
3 users	Logged-in users
Load average	1, 5, 15-min load

5. Command: **kill** (Terminate Processes)

kill sends signals to processes (usually to stop them).

Why **kill** is Used

Purpose	Explanation
Stop unresponsive processes	Forced termination
Stop background jobs	Clean up resources
Send custom signals	Restart, terminate, pause processes

Syntax

kill [signal] PID

Important Signals

Signal	Number	Meaning
SIGTERM	15	Graceful stop
SIGKILL	9	Force stop
SIGSTOP	19	Pause process
SIGCONT	18	Resume stopped process

Examples

Command	Explanation
kill 1234	Gracefully stop PID 1234
kill -9 1234	Force kill
kill -SIGSTOP 1234	Pause process
kill -SIGCONT 1234	Resume process

Summary Table

Command	Meaning
kill PID	Stop process

kill -9	Force kill
kill -SIGSTOP	Pause
kill -SIGCONT	Resume

6. Command: **netstat** (Network Statistics)

netstat displays network connections, routing tables, and ports.

Why **netstat** is Used

Purpose	Explanation
Check active connections	Identify open ports
Check listening services	Verify if services are running
View routing table	Networking troubleshooting
Monitor network stats	Performance checks

Syntax

netstat [options]

Important Options

Option	Meaning
netstat -tulnp	Show open ports with PID
netstat -rn	Routing table
netstat -a	All connections

Examples

Command	Explanation
netstat -tulnp	Shows listening ports and services
netstat -rn	Show routing table

Summary Table

Command	Meaning
netstat -tulnp	Ports + PID
netstat -rn	Routing table
netstat -a	All connections

7. Command: **ip** (Modern Network Management)

ip is the modern replacement for **ifconfig**, used to manage network interfaces.

Why **ip** is Used

Purpose	Explanation
---------	-------------

View interfaces	Detailed network information
Configure addresses	Assign IPs
Manage routes	Networking tasks
Replace older tools	Modern and powerful

Syntax

ip [object] [command]

Important Objects

Object	Meaning
ip a	Show IP addresses
ip link	Show interfaces
ip route	Routing table

Examples

Command	Explanation
ip a	Show all interface details
ip route	Show routing configuration
ip link set eth0 down	Disable interface

Summary Table

Command	Meaning
ip a	IP information
ip route	Routing table
ip link	Interface control

8. Command: **ss** (Socket Statistics)

ss is a faster, modern replacement for netstat.

Why **ss** is Used

Purpose	Explanation
Check connections	Faster than netstat
Check listening ports	More accurate
Analyze sockets	Used by network engineers

Syntax

ss [options]

Important Options

Option	Meaning
ss -tuln	Show listening ports
ss -tnp	Show TCP connections with PID

Examples

Command	Explanation
ss -tuln	Listening TCP/UDP ports
ss -tnp	Connections with process info

Summary Table

Command	Meaning
ss -tuln	Listening ports
ss -tnp	TCP + PID info

9. Command: **systemctl** (Service & System Manager)

systemctl controls system services and daemons on systemd-based Linux systems.

Why **systemctl** is Used

Purpose	Explanation
Start/stop services	Service management
Enable/disable services	Boot-time control
Check service status	Troubleshooting
Manage entire system targets	Shutdown, reboot, rescue mode

Syntax

systemctl [command] service

Important Commands

Command	Meaning
systemctl start service	Start service
systemctl stop service	Stop service
systemctl restart service	Restart
systemctl status service	Check status
systemctl enable service	Start at boot
systemctl disable service	Disable at boot

Examples

Command	Explanation
systemctl start nginx	Start nginx service

systemctl status sshd	Check SSH service
systemctl enable docker	Enable Docker at boot

Summary Table

Command	Meaning
systemctl start	Start service
systemctl stop	Stop service
systemctl restart	Restart
systemctl status	Status
systemctl enable	Enable at boot
systemctl disable	Disable at boot

1. Command: **tar** (Archive and Compress Files)

tar is used to **create, extract, compress, and manage archives**.
It is the most common tool for packaging multiple files into a single archive.

Why **tar** is Used

Purpose	Explanation
Create backups	Combine and compress files
Extract archives	Unpack .tar, .tar.gz, .tgz
Transfer large folders	Package into a single file
Compress data	Save space

Syntax

tar [options] archive_name files_or_directories

Important Options

Option	Meaning
-c	Create archive
-x	Extract archive
-v	Verbose (show progress)
-f	Specify archive filename
-z	Compress with gzip (.gz)
-j	Compress with bzip2 (.bz2)
-t	List archive contents

(Options are typically combined.)

Examples

Command	Explanation
---------	-------------

tar -cvf backup.tar folder/	Create archive
tar -xvf backup.tar	Extract archive
tar -czvf backup.tar.gz folder/	Create gzip-compressed archive
tar -xzvf backup.tar.gz	Extract gzip archive
tar -tf archive.tar	List contents

Summary Table

Command	Meaning
tar -cvf	Create archive
tar -xvf	Extract archive
tar -czvf	Create compressed archive
tar -xzvf	Extract compressed archive
tar -tf	List contents

2. Command: **ssh** (Secure Shell Login)

ssh is used to securely connect to remote Linux machines.

Why **ssh** is Used

Purpose	Explanation
Remote login	Access remote servers
Remote command execution	Run commands without logging in fully
Secure data exchange	Encrypted connection
Administration	Manage cloud servers

Syntax

ssh username@host

Important Options

Option	Meaning
-p	Custom port
-i	Use identity/private key
ssh user@ip command	Run single remote command

Examples

Command	Explanation
ssh sahil@192.168.1.10	Login to remote machine
ssh -p 2222 user@server	Connect using port 2222
ssh -i key.pem user@server	Connect using SSH key
ssh user@server "uptime"	Run a single command

Summary Table

Command	Meaning
ssh user@host	Remote login
ssh -p	Use specific port
ssh -i	Use private key
ssh user@host "cmd"	Run remote command

3. Command: **scp** (Secure Copy)

scp copies files **between local and remote systems** securely.

Why **scp** is Used

Purpose	Explanation
Transfer files	Upload/download securely
Copy whole directories	Using -r
Alternative to FTP	More secure

Syntax

scp [options] source destination

Important Options

Option	Meaning
-r	Copy directories
-P	Specify port
-i	Identity key

Examples

Command	Explanation
scp file.txt user@server:/path/	Upload file
scp user@server:/path/file.txt .	Download file
scp -r folder user@server:/backup/	Upload directory
scp -P 2222 file user@server:/path/	Use custom port

Summary Table

Command	Meaning
scp file user@host:	Upload file
scp user@host:file .	Download file
scp -r	Copy directory
scp -P	Custom port

4. Command: **ping** (Check Network Connectivity)

ping is used to test **connectivity between systems**.

Why ping is Used

Purpose	Explanation
Test connection	Check if host is reachable
Measure latency	Round-trip time
Troubleshoot network	Packet loss detection

Syntax

ping host

Important Options

Option	Meaning
-c N	Send N packets
-i	Set interval
-w	Timeout

Examples

Command	Explanation
ping google.com	Test connection
ping -c 4 8.8.8.8	Send only 4 packets
ping -w 5 google.com	Stop after 5 seconds

Summary Table

Command	Meaning
ping host	Connectivity test
ping -c	Packet count
ping -w	Timeout

5. Command: **chmod** (Advanced Permissions)

chmod controls **read, write, execute** permissions using numeric and symbolic modes.

Permission Basics

Permission	Number	Symbol
Read	4	r
Write	2	w
Execute	1	x

Why Advanced **chmod** is Used

Purpose	Explanation
Set correct security	Prevent unauthorized access
Make scripts executable	Needed for shell scripts
Permission troubleshooting	Fix app or server issues

Numeric Permission Examples

Command	Meaning
chmod 755 file	Owner full; others read+execute
chmod 644 file	Owner read+write; others read
chmod 700 file	Private file (owner only)

Symbolic Permission Examples

Command	Explanation
chmod u+x script.sh	Add execute to owner
chmod g-w file	Remove write from group
chmod o-r file	Remove read from others
chmod a+r file	Add read to everyone

Summary Table

Command	Meaning
chmod 755	Script permission
chmod 644	Normal file permission
chmod u+x	Add execute
chmod g-w	Remove write

6. Command: **sed** (Stream Editor)

sed edits text **non-interactively**, line by line.

Why **sed** is Used

Purpose	Explanation
Search & replace	Modify text patterns
Delete lines	Clean up logs or files
Insert or append text	Automate file edits
Powerful automation	Scripting and DevOps tasks

Syntax

sed [options] 'command' file

Important Operations

Operation	Meaning
s/old/new/	Replace text
d	Delete lines
p	Print lines
i	Insert before line
a	Append after line

Examples

Command	Explanation
sed 's/apple/orange/g' file	Replace all “apple” with “orange”
sed '5d' file	Delete line 5
sed -n '3p' file	Print line 3
sed '3i text' file	Insert before line 3
sed '3a text' file	Append after line 3

Summary Table

Command	Meaning
s/old/new/g	Replace
d	Delete
p	Print
i	Insert
a	Append

7. Command: **awk** (Pattern Scanning and Processing)

awk processes data **column-wise** and is extremely powerful for analyzing text files.

Why **awk** is Used

Purpose	Explanation
Process columns	Extract fields from logs, CSV, etc.
Perform calculations	Sum, average, counts
Filter data	Match patterns
Generate reports	System and log analysis

Syntax

```
awk 'pattern { action }' file
```

Basic **AWK** Concepts

Symbol	Meaning
--------	---------

\$1, \$2, \$3	Columns
\$0	Entire line
NR	Line number
FS	Field separator

Examples

Command	Explanation
awk '{print \$1}' file	Print first column
awk '{print \$1, \$3}' file	Print columns 1 and 3
awk '/error/' file	Print lines containing "error"
awk '{sum += \$2} END {print sum}' file	Sum second column

Summary Table

Command	Meaning
print \$1	Print first field
/pattern/	Match lines
sum += \$2	Add values
END {}	Final output

8. Command: **crontab** (Scheduled Tasks)

crontab schedules commands to run automatically at specific times.

Why **crontab** is Used

Purpose	Explanation
Automate tasks	Backups, monitoring, scripts
Time-based triggers	Daily/weekly/monthly
Server maintenance	Log rotation, cleanup

Syntax

crontab -e

Crontab Time Fields

Field	Meaning
Minute	0–59
Hour	0–23
Day of month	1–31
Month	1–12
Day of week	0–6

Examples

Schedule	Meaning
0 2 * * * backup.sh	Run every day at 2 AM
* /5 * * * * script.sh	Run every 5 minutes
0 * /6 * * * job.sh	Run every 6 hours
0 0 * * 0 cleanup.sh	Run every Sunday

Commands

Command	Meaning
crontab -e	Edit cron jobs
crontab -l	List cron jobs
crontab -r	Remove all cron jobs

Chapter 1: File Handling & Directories (Short + Detailed Notes)

1. Linux Filesystem Basics

- Linux uses a **hierarchical tree structure** starting from / (root).
- Everything is treated as a **file** (text files, directories, devices, processes).
- Two path types:
 - **Absolute path** → starts from /
 - **Relative path** → based on current location

2. Important Directories

- / → root of entire filesystem
- /home → user home directories
- /etc → system configuration files
- /bin, /sbin → essential system commands
- /usr → user programs, libraries
- /var → logs, cache, data that changes
- /tmp → temporary files
- /dev → device files
- /proc, /sys → virtual system info

3. Directory Navigation

- . → current directory
- .. → parent directory
- ~ → home directory
- cd moves between directories (absolute or relative).
- pwd shows your current location.

4. Creating & Managing Directories

- Create folder: mkdir
 - mkdir folder → make directory
 - mkdir -p a/b/c → create nested structure

- Remove folder: rmdir (empty only)
 - rmdir folder
 - rmdir -p path/sub1/sub2

5. File Creation & Viewing

- touch file → create empty file or update timestamp
- cat file → view file
 - cat -n file → show line numbers
 - cat file1 file2 > merge.txt → merge files

6. File Copy, Move, Delete

- Copy: cp
 - cp file dest/
 - cp -r dir dest/ → copy directories
 - cp -p → preserve permissions, timestamps
- Move/Rename: mv
 - mv old new → rename
 - mv file dir/ → move
 - mv -i → confirm overwrite
- Delete: rm
 - rm file → remove file
 - rm -r dir → remove directory
 - rm -rf → force & recursive (dangerous)

7. Searching & Locating Files

- find → search in directory tree
 - find . -name "*.txt"
 - find /var -type f -size +100M
 - find . -delete
- grep → search text inside files
 - grep "error" logfile
 - grep -i → ignore case
 - grep -r → recursive search

8. Permissions & Ownership

- Permissions: **r** (read), **w** (write), **x** (execute)

- Numeric format:
 - 755 (owner full, others read+execute)
 - 644 (owner read/write, others read)
- Change permissions: chmod
 - chmod 755 file
 - chmod u+x script.sh
- Change owner: chown
 - chown user file
 - chown user:group file
 - chown -R for recursive

9. Archiving & Compression

- tar creates/extracts archives
 - tar -cvf file.tar folder/
 - tar -xvzf file.tar.gz
 - tar -tf file.tar → list contents

10. Directory & File Size Checks

- Disk usage per filesystem: df -h
- Size of folders/files: du -sh folder

Chapter 2: Nano & Vi/Vim Editors (Short + Detailed Notes)

1. Nano Editor

Create a File

nano example.txt

Saving & Exiting

- Press **Ctrl + X** → exit prompt
- Press **Y** → save changes
- Press **Enter** → confirm filename

Nano is simple, beginner-friendly, and works directly with normal typing.

2. Vi / Vim Editor

Vi/Vim has **modes**, so the keyboard behaves differently in each mode.

Modes of Vim

1. **Command Mode (default)** – navigate, delete, copy, paste
2. **Insert Mode** – type text
3. **Execution Mode** – run commands using :
4. **Visual Mode** – select text

Note: Press Esc to return to Command Mode from any mode.

2.1 Command Mode (Default Mode)

Used for navigation + editing actions.

Delete

- dd → delete current line
- dw → delete current word
- ndd → delete *n* lines

Copy (Yank)

- yy → copy current line
- yw → copy word
- nyy → copy *n* lines

Cut (Change)

- cc → cut line + enter Insert Mode
- cw → cut word + enter Insert Mode
- ncc → cut *n* lines

Paste

- P → paste above cursor (or where cursor is)

Undo / Redo

- u → undo
- Ctrl + r → redo

Movement

- G / L → end of file
- gg / H → beginning of file
- ngg → go to line *n*

2.2 Insert Mode

Enter Insert Mode using:

- i → insert at cursor
- o → new line below
- O → new line above

(Press **Esc** to go back.)

2.3 Execution Mode

Activated with : (colon).

Quit / Save

- :q → quit (no save)
- :q! → quit without saving (force)
- :w → save
- :wq / :x → save + quit
- :wq! → force save + quit

Display Settings

- :set nu → show line numbers
- :set nonu → hide line numbers

Search

- :/word → search for *word*

Find & Replace

:%s/old/new/g

(Replaces all occurrences.)

Run Terminal Commands

!:command

2.4 Visual Mode

Used to select text before editing.

- v → select character-wise
- V → select line-wise
- Ctrl + v → select block/column

Actions on Selection

- y → copy
- d → delete
- c → cut (enter Insert Mode)
- P → paste

Chapter 3: Users & Groups in Linux (Short + Detailed Notes)

1. Types of Users

1. Root User (UID = 0)

- Superuser; full control over system.
- Can modify/delete any file, install software, manage users, change permissions.

Commands

```
sudo -i      # switch to root shell
sudo <command> # run command as root
```

2. System Users (UID 1–999)

- Created by the system for running services (Apache, MySQL, SSH, etc.).
- No login shell by default.
- Limited access for security.

3. Local Users (UID 1000–65535)

- Normal users for daily tasks.
- Each gets a home directory: /home/username.
- Limited access unless elevated with sudo.

2. User Management

Create User

```
adduser tony      # recommended, creates home dir
useradd username  # no home dir
useradd -m username # create home dir
```

Set or Change Password

```
passwd username
```

Delete User

```
userdel -r username # remove user + home dir
```

List All Users

```
cat /etc/passwd
```

3. User Info Files

/etc/passwd

Stores user account info.

Example:

```
steve:x:1001:1001:Steve User:/home/steve:/bin/bash
```

Fields

1. Username
2. Password placeholder (x)

3. UID
4. GID
5. User info (comment)
6. Home directory
7. Login shell

/etc/shadow

Stores password hashes + password policy.

Fields

1. Username
2. Encrypted password
3. Last password change
4. Min days
5. Max days
6. Warning days
7. Inactive days
8. Expiry
9. Reserved

4. Password Policy (chage)

Command	Description
chage -l username	Show password aging
chage -M 30 username	Max 30 days
chage -m 7 username	Min 7 days
chage -W 5 username	Warn 5 days before expiry
chage -I 10 username	Inactive 10 days after expiry
chage -E 2025-07-01 username	Set expiry date

5. Group Management

Create Group

groupadd avengers

Group Info Files

/etc/group

Fields:

1. Group name
2. Password placeholder
3. GID
4. Members

/etc/gshadow

Fields:

1. Group name
2. Encrypted password
3. Group admins
4. Group members

6. Manage Group Membership

Add User to Group

`usermod -aG avengers natasha`

`gpasswd -a steve avengers`

Remove User from Group

`gpasswd -d steve avengers`

Add Multiple Users (Overwrites old list)

`gpasswd -M steve,thor,bruce avengers`

Assign Group Admin

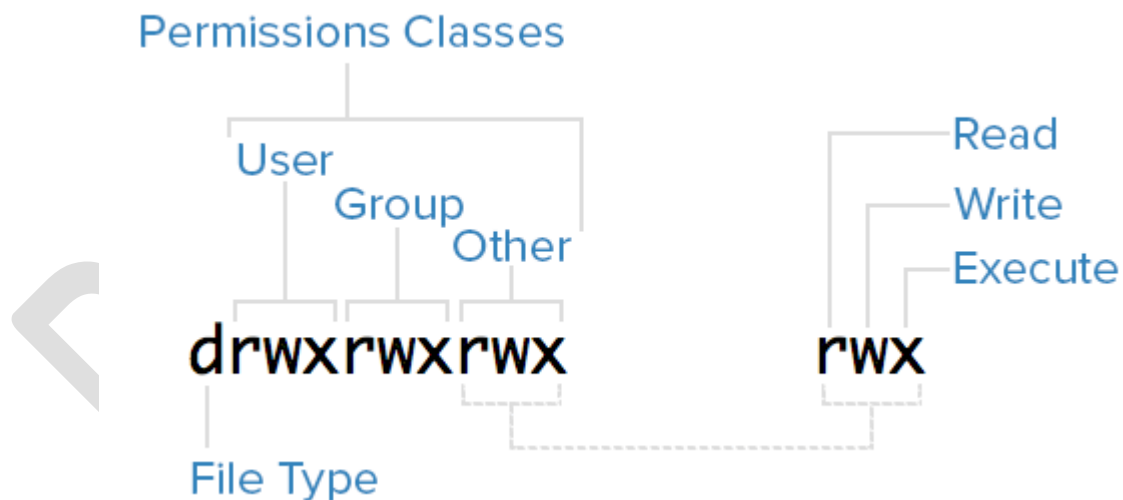
`gpasswd -A steve avengers`

Remove Group Admin

`gpasswd -A '' avengers`

Delete Group

`groupdel -f avengers`



Chapter 4: Linux File Permissions (Short + Detailed Notes)

1. Introduction

Linux file permissions are a core part of the system's **security model**. They define **who can read, write, or execute** a file or directory.

Permissions protect:

- System files
- User data
- Applications
- Services

Every file and directory in Linux has:

- **Owner**
- **Group**
- **Permission set**

2. Permission Categories

Linux permissions are defined for **three entities**:

Category	Meaning
User (u)	Owner of the file
Group (g)	Group associated with the file
Others (o)	Everyone else

3. Permission Types

Each category can have the following permissions:

Permission	Symbol	Value
Read	r	4
Write	w	2
Execute	x	1

4. Viewing File Permissions

```
ls -l
```

Example output:

```
-rwxr-xr-- 1 sahil devops 1024 file.sh
```

Breakdown:

```
- rwx r-x r--
|   |   |   |
|   |   |   |---- Others
|   |   |   |---- Group
|   |   |   |---- User
|   |   |   |---- File type
```

5. File Type Indicator

Symbol	Meaning
-	Regular file
d	Directory
l	Symbolic link
c	Character device
b	Block device

6. Permissions on Files vs Directories

Files

Permission	Effect
Read (r)	View file contents
Write (w)	Modify file
Execute (x)	Run file as program

Directories

Permission	Effect
Read (r)	List directory contents
Write (w)	Create/delete files
Execute (x)	Enter directory

⚠ **Directory execute (x) is mandatory** to access files inside.

7. Changing Permissions — `chmod`

Symbolic Mode

```
chmod u+x file.sh
chmod g-w file.sh
chmod o=r file.sh
```

Examples:

```
chmod u+rw file.txt
chmod go+rx file.txt
chmod a+r file.txt
```

Numeric (Octal) Mode 🔥 MOST IMPORTANT

Permission	Value
r	4
w	2
x	1

Add values:

Mode	Meaning
7	rwX
6	rw-
5	r-X
4	r--
0	---

Example:

```
chmod 755 script.sh
```

Meaning:

```
User    → 7 → rwX
Group   → 5 → r-X
Others  → 5 → r-X
```

8. Common Permission Examples

Command	Meaning
chmod 644 file	Owner RW, others read
chmod 755 dir	Full for owner, read+execute for others
chmod 600 file	Private file
chmod 777 file	❌ Unsafe (avoid)

9. Changing Ownership — `chown`

Change owner

```
sudo chown sahil file.txt
```

Change owner and group

```
sudo chown sahil:devops file.txt
```

Recursive ownership

```
sudo chown -R sahil:devops /project
```

10. Changing Group — `chgrp`

```
sudo chgrp devops file.txt
```

11. Recursive Permission Changes

```
chmod -R 755 /opt/app
```

⚠ Use with caution — can break systems.

12. Default Permissions & `umask`

`umask` defines **default permission subtraction**.

Check:

```
umask
```

Example:

```
022
```

Defaults:

- Files: `666 - umask`
- Directories: `777 - umask`

With `022`:

- Files → `644`

- Directories → 755

13. Special Permissions (Advanced)

SUID (Set User ID)

```
chmod u+s file
```

Shown as:

```
-rwsr-xr-x
```

Runs file with **owner's** privileges.

SGID (Set Group ID)

```
chmod g+s dir
```

Files inherit group ownership.

Sticky Bit

```
chmod +t /shared
```

Shown as:

```
drwxrwxrwt
```

Only file owner can delete files (used in `/tmp`).

14. Permission Precedence Rule

- User permissions
- Group permissions
- Others permissions

Linux checks **in this exact order**.

15. Security Best Practices

- Never use 777
- Use groups for shared access
- Apply least privilege principle
- Audit permissions regularly
- Protect config files with 600

16. Interview & Exam Key Points

- Permissions = rwx model
- Numeric mode uses octal values
- Directories need execute permission to access
- `chmod`, `chown`, `chgrp` are core commands
- Special permissions add extra security logic

17. Quick Revision Summary

- View → `ls -l`
- Change permissions → `chmod`
- Change owner → `chown`
- Change group → `chgrp`
- Default permissions → `umask`

Practice Tasks

1. Create a file and set permission 640
2. Create a directory with permission 750
3. Assign ownership to another user
4. Test access with different users
5. Apply sticky bit on a directory

ACL (Access Control Lists)

1. What is ACL?

ACL (Access Control List) provides **fine-grained permissions** beyond the traditional **user-group-others** model.

With ACL:

- Multiple users can have permissions on the same file
- Multiple groups can have permissions on the same file

ACL is mainly used when:

- One file needs access by multiple teams
- Group-based permissions are not enough

2. Why ACL is Needed

Traditional permissions limitation:

User | Group | Others

ACL allows:

```
User1 → rw
User2 → r
Group1 → rw
Group2 → r
```

3. Check ACL Support

```
mount | grep acl
```

Check filesystem:

```
tune2fs -l /dev/sda1 | grep ACL
```

4. Viewing ACL

```
getfacl file.txt
```

Example output:

```
# file: file.txt
# owner: sahil
# group: devops
user::rw-
user:rahul:r--
group::r--
mask::rw-
other::r--
```

5. Setting ACL

Grant permission to a user

```
setfacl -m u:rahul:rw file.txt
```

Grant permission to a group


```
setfacl -m g:test:r file.txt
```

6. Remove ACL

Remove ACL for specific user

```
setfacl -x u:rahul file.txt
```

Remove all ACLs

```
setfacl -b file.txt
```

7. Default ACL (Important for Directories)

Files created inside directory inherit ACLs.

```
setfacl -d -m u:rahul:rwX /shared
```

8. ACL Mask (Very Important)

ACL **mask** defines the **maximum effective permission**.

Even if:

```
user:rahul:rwX
```

But mask is:

```
mask::r--
```

👉 Effective permission = **r--**

Modify mask:

```
setfacl -m m:rwX file.txt
```

umask (Deep Dive)

9. What is umask?

umask defines **default permission removal** for newly created files and directories.

It does **NOT** grant permissions — it removes permissions.

10. Default Permission Logic

Type	Base Permission
File	666
Directory	777

Final permission = **Base** – **umask**

11. Checking umask

umask

Example:

022

12. Permission Calculation Example

With umask 022:

Files:

666

-022

644

Directories:

777

-022

755

13. Setting umask Temporarily

umask 027

14. Setting umask Permanently

User level:

~/.bashrc

~/.profile

System-wide:

/etc/profile

/etc/bashrc

15. Root vs Normal User umask

User	Typical umask
Root	022
Normal User	002 or 022

PART 3: Users, Groups & Permissions – Combined Labs

LAB 1: Group-Based Access

groupadd devteam

useradd dev1

usermod -aG devteam dev1

Directory setup:

mkdir /project

chown :devteam /project

chmod 770 /project

LAB 2: Shared Directory with SGID

chmod g+s /project

Result:

- All files inherit group devteam

LAB 3: ACL-Based Access

```
setfacl -m u:testuser:rx /project
```

Now:

- testuser can access without being in group

LAB 4: Sticky Bit Protection

```
chmod +t /shared
```

Only owner can delete files.

LAB 5: umask Testing

```
umask 027
```

```
touch file1
```

```
mkdir dir1
```

```
ls -l
```

Observe default permissions.

PART 4: Linux Permissions – Interview Q&A

Q1. Difference between chmod 755 and chmod 777?

- 755 → Controlled access
- 777 → Everyone full access (security risk)

Q2. Why ACL is better than chmod?

- Supports multiple users & groups
- Fine-grained control

Q3. What is SGID on directory?

- New files inherit group ownership

Q4. What is umask?

- Default permission subtraction mask

Q5. What happens if directory has no execute permission?

- Cannot access files inside

Q6. Where is ACL stored?

- As filesystem metadata (not visible via ls)

Q7. What does “+” mean in ls -l?

-rw-rwxr--+

- Indicates ACL exists

Q8. Which has higher priority: ACL or chmod?

- ACL overrides standard permissions

Q9. How to remove all ACLs?

setfacl -b file

Q10. Real-world ACL use case?

- Multiple teams accessing shared project folders

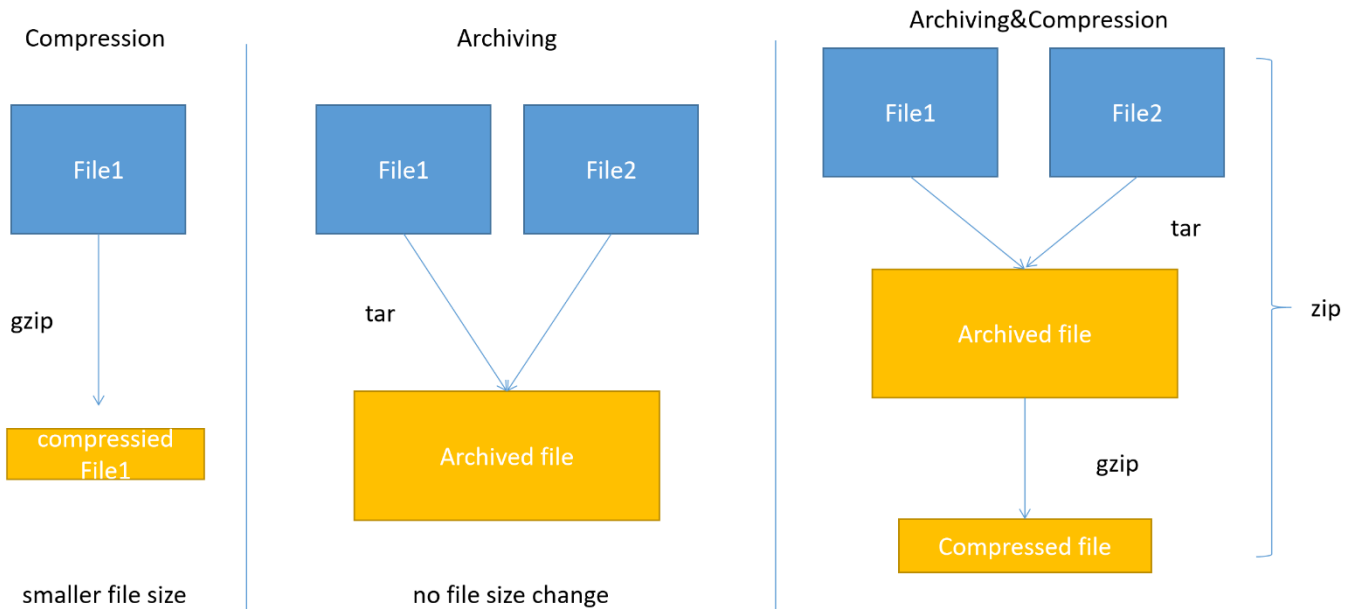
Final Pro Tips (Industry Standard)

- Use **groups first**
- Use **ACL only when needed**
- Avoid 777
- Audit permissions regularly
- Document ACL usage

Quick Master Summary

- **chmod** → basic permissions
- **ACL** → advanced permissions
- **umask** → default permission control
- **SGID + Sticky** → shared environments

Chapter 5: Archiving & Compression (Short + Detailed Notes)



1. Concepts

Archiving

- Means **combining multiple files/folders into one file**.
- Does **not** reduce size; just packs everything together.

Compression

- Reduces file size.
- Used with gzip, bzip2, xz, zip, etc.

2. Common tar Options

Option	Meaning
c	create archive
v	verbose (show progress)
f	filename
z	gzip compression
j	bzip2 compression
J	xz compression
x	extract archive
C	extract at custom location

3. Check Folder Size

du -sh /etc

4. Gzip Compression (.tar.gz)

Create Backup

`tar -cvzf backup.tar.gz /etc`

Extract

`tar -xvzf backup.tar.gz -C /opt`

5. bzip2 Compression (.tar.bz2)

Create Backup

`tar -cvjf backup.tar.bz2 /etc`

Extract

`tar -xvjf backup.tar.bz2 -C /opt`

6. xz Compression (.tar.xz)

Create Backup

`tar -cvJf backup.tar.xz /etc`

Extract

`tar -xvJf backup.tar.xz -C /opt`

7. ZIP Compression

Create ZIP

`zip -r backup.zip /etc`

Extract ZIP

`unzip backup.zip -d /opt`

Linux tar Command

CREATE

-cf

-czf

-cjf

LIST

-t or --list

EXTRACT

-xf

-xzf

-xjf

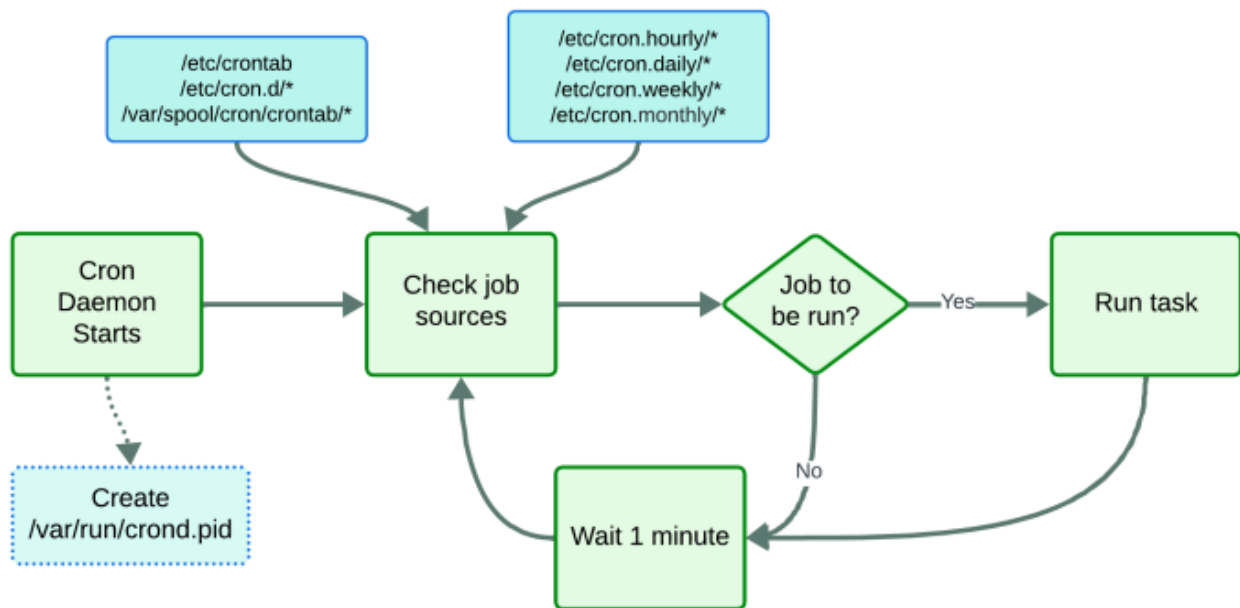
ADD/REMOVE

--append or -r

--delete

Chapter 6: Crontab (Task Scheduling)

1. Introduction to Task Scheduling



In Linux, **task scheduling** allows commands or scripts to be executed automatically at predefined times or intervals. This functionality is provided by the **cron service**, which runs as a background daemon.

Task scheduling is essential for:

- System maintenance
- Log cleanup
- Backups
- Automation tasks
- Monitoring and reporting

2. What is Cron?

Cron is a time-based job scheduler in Unix-like operating systems.

- Runs as a daemon: `crond`
- Reads scheduling instructions from **crontab files**
- Executes commands automatically without user interaction

Check cron service status:

```
systemctl status cron      # Ubuntu/Debian
systemctl status crond     # RHEL/CentOS
```

3. What is Crontab?

A **crontab (cron table)** is a file that contains a list of scheduled jobs for a user.

Each user can have their own crontab file.

Edit user crontab:

```
crontab -e
```

List crontab:

```
crontab -l
```

Remove crontab:

```
crontab -r
```

4. Crontab File Structure

A cron job consists of **six fields**:

```
* * * * * command
| | | | |
| | | | └─ Day of week (0-7) [Sunday = 0 or 7]
| | | └─── Month (1-12)
| | └───── Day of month (1-31)
| └──────── Hour (0-23)
└────────── Minute (0-59)
```

5. Time Field Syntax

Field	Allowed Values
Minute	0-59
Hour	0-23
Day of Month	1-31
Month	1-12 or Jan-Dec

Day of Week	0–7 or Sun–Sat
-------------	----------------

6. Special Characters in Crontab

Symbol	Meaning
*	Every value
,	Multiple values
-	Range
/	Step values

Examples:

```
* /5 * * * * # Every 5 minutes
0 9 * * * # Every day at 9 AM
0 0 * * 0 # Every Sunday at midnight
```

7. Common Cron Job Examples

Run a script every day at 2 AM

```
0 2 * * * /home/user/backup.sh
```

Clean temporary files every Sunday

```
0 3 * * 0 rm -rf /tmp/*
```

Restart service daily

```
0 1 * * * systemctl restart nginx
```

8. System-Wide Cron Directories

Linux also supports system cron jobs:

Directory	Purpose
/etc/crontab	System crontab
/etc/cron.hourly/	Runs every hour
/etc/cron.daily/	Runs daily
/etc/cron.weekly/	Runs weekly

/etc/cron.monthly/	Runs monthly
--------------------	--------------

Example:

```
ls /etc/cron.daily
```

9. Difference: User Crontab vs System Crontab

Feature	User Crontab	System Crontab
User field	✗ No	✓ Yes
Location	User-specific	/etc/crontab
Permissions	User level	Root level

System crontab format:

```
* * * * * user command
```

10. Environment in Cron Jobs (IMPORTANT)

Cron runs with a **minimal environment**.

Best practices:

- Use **absolute paths**
- Define shell explicitly

Example:

```
SHELL=/bin/bash
PATH=/usr/bin:/bin
```

11. Logging and Debugging Cron Jobs

Check cron logs:

```
grep CRON /var/log/syslog      # Ubuntu
grep CRON /var/log/cron        # RHEL
```

Redirect output:

```
0 2 * * * /script.sh >> /var/log/script.log 2>&1
```

12. Permissions and Security

- Only authorized users can use cron
- Controlled by:
 - `/etc/cron.allow`
 - `/etc/cron.deny`

If both exist:

- `cron.allow` takes priority

13. At vs Cron (Comparison)

Feature	cron	at
Execution	Repeated	One-time
Scheduling	Periodic	Single run
Usage	Automation	Ad-hoc jobs

Example:

```
echo "reboot" | at 02:00
```

14. Best Practices (Industry Standard)

- Always log cron output
- Avoid running heavy jobs at peak hours
- Use scripts instead of inline commands
- Test scripts manually before scheduling
- Document cron jobs

15. Interview & Exam Points

- Cron is daemon-based
- Uses time fields + command
- Runs with limited environment
- Logs stored in `/var/log`
- Supports per-user scheduling

Quick Revision Summary

- `crontab -e` → edit jobs
- `crontab -l` → list jobs
- `crontab -r` → remove jobs
- `* * * * *` → scheduling format
- `/etc/cron.*` → system jobs

Batch-B60

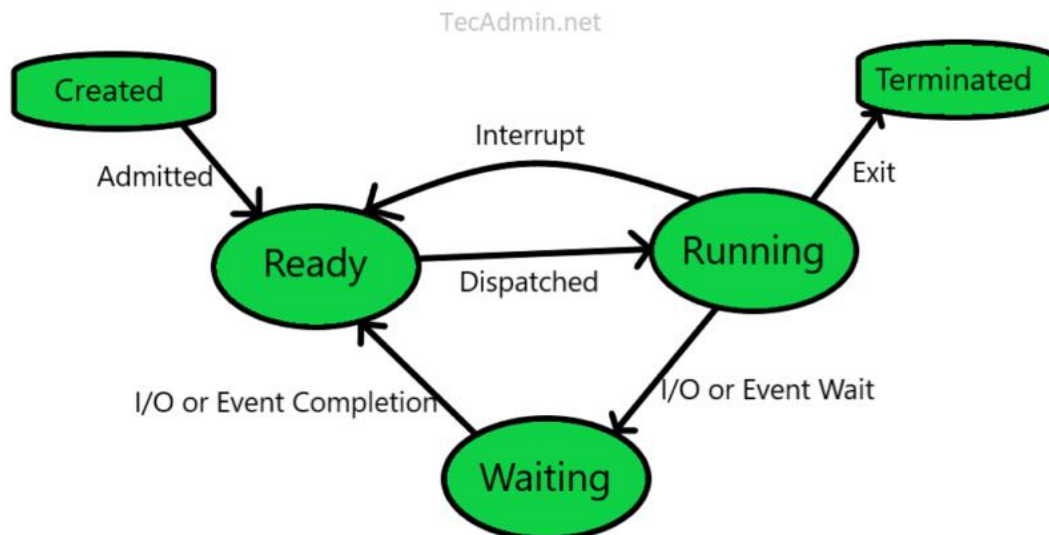
Chapter 7: Process Management

1. Introduction

A **process** is an instance of a running program.

Linux is a **multitasking operating system**, capable of running thousands of processes simultaneously.

Linux Process States



Process management involves:

- Creating processes
- Monitoring processes
- Controlling execution
- Managing priorities
- Terminating processes

2. Process vs Program

Program	Process
Static file on disk	Active execution in memory
Not running	Running instance

Example: /bin/bash	Bash shell session
--------------------	--------------------

3. Process Identifiers

Each process has:

- **PID (Process ID)** → Unique identifier
- **PPID (Parent Process ID)** → Creator process
- **UID** → Owner user

```
ps
echo $$
```

4. Types of Processes

4.1 Foreground Process

- Runs interactively
- Attached to terminal

```
sleep 100
```

4.2 Background Process

- Runs without blocking terminal

```
sleep 100 &
```

4.3 Daemon Process

- Runs in background continuously
- No terminal attached

Examples:

- sshd
- cron
- systemd

5. Process States

Linux process states:

State	Meaning
-------	---------

R	Running / Runnable
S	Sleeping
D	Uninterruptible sleep
T	Stopped
Z	Zombie

Check:

```
ps aux
```

6. Viewing Processes

ps command

```
ps
ps -ef
ps aux
```

Key fields:

- PID
- CPU
- MEM
- COMMAND

top command

```
top
```

Interactive:

- **P** → Sort by CPU
- **M** → Sort by memory
- **k** → Kill process

htop (advanced)

```
htop
```

(Colorful, interactive – widely used in production)

7. Process Hierarchy

Linux processes form a **tree structure**.

View hierarchy:

```
pstree
pstree -p
```

8. Controlling Processes

Stop a process

```
Ctrl + Z
```

Resume in foreground

```
fg
```

Resume in background

```
bg
```

List jobs:

```
jobs
```

9. Killing Processes

kill command

```
kill PID
```

Signals

Signal	Number	Meaning
SIGTERM	15	Graceful stop
SIGKILL	9	Force kill
SIGSTOP	19	Pause

Example:

```
kill -9 1234
```

killall

```
killall nginx
```

pkill

```
pkill httpd
```

10. Process Priority (nice & renice)

Linux uses **priority scheduling**.

Priority range:

-20 (highest) → 19 (lowest)

Start process with priority

```
nice -n 10 sleep 200
```

Change running process priority

```
renice -5 -p 1234
```

11. CPU & Memory Monitoring

CPU usage

```
top  
mpstat
```

Memory usage

```
free -h  
vmstat
```

12. Process Filesystem (/proc)

Linux exposes process info via virtual filesystem.

Example:

```
ls /proc/1234  
cat /proc/1234/status
```

13. Background Execution Techniques

nohup

```
nohup script.sh &
```

Runs even after logout.

disown

`disown`

Removes job from shell control.

14. Zombie & Orphan Processes

Zombie

- Process finished execution
- Parent hasn't collected exit status

Shown as:

Z

Orphan

- Parent dies
- Adopted by `systemd`

15. Managing Services (systemd)

Check service

```
systemctl status nginx
```

Start / Stop / Restart

```
systemctl start nginx
systemctl stop nginx
systemctl restart nginx
```

16. Signals (Advanced Concept)

List signals:

```
kill -l
```

Send signal:

```
kill -SIGTERM PID
```

17. Best Practices (Industry Standard)

- Use `SIGTERM` before `SIGKILL`

- Monitor CPU/memory regularly
- Avoid running apps as root
- Use `systemd` for services
- Investigate zombie processes

18. Interview & Exam Key Points

- Process $\neq$ program
- PID uniquely identifies process
- Zombie = dead but not cleaned
- `nice` controls priority
- `/proc` provides runtime info

19. Quick Revision Summary

- View $\rightarrow$ `ps`, `top`
- Control $\rightarrow$ `fg`, `bg`, `jobs`
- Kill $\rightarrow$ `kill`, `pkill`
- Priority $\rightarrow$ `nice`, `renice`
- Services $\rightarrow$ `systemctl`

Practice Tasks (Highly Recommended)

1. Start a background process
2. Change its priority
3. Kill it gracefully
4. Force kill it
5. Inspect `/proc/PID`

Chapter 8: Networking

1. Introduction to Linux Networking

Linux is inherently a **network-oriented operating system**. Its networking framework supports multi-host communication, remote administration, and internet connectivity. Network management in Linux involves configuring interfaces, managing IP addressing, routing, name resolution, and securing traffic.

Modern Linux networking primarily uses the **ip utility** (from the `iproute2` suite), replacing older tools such as `ifconfig`, `route`, and `netstat`.

2. Network Interfaces

Definition

A network interface represents a hardware or virtual connection to a network segment. Common interface types include:

- Physical: `eth0`, `ens33`
- Loopback: `lo`
- Virtual: `docker0`, `veth*`

List Interfaces

```
ip link show
```

Output displays:

- Interface name
- Status (UP/DOWN)
- MAC address

Enable/Disable Interface

```
sudo ip link set eth0 up
sudo ip link set eth0 down
```

3. IP Address Management

View Assigned Addresses

```
ip addr show
```

Assign an IPv4 Address

Temporary (not persistent across reboot):

```
sudo ip addr add 192.168.1.10/24 dev eth0
```

Remove an Address

```
sudo ip addr del 192.168.1.10/24 dev eth0
```

4. Static vs. Dynamic IP Addressing

Dynamic IP

DHCP assigns addresses automatically:

```
sudo dhclient eth0
```

Static IP Configuration

On modern Ubuntu systems, network settings are configured via **Netplan**.

Example configuration file: `/etc/netplan/01-netcfg.yaml`

```
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: no
      addresses: [192.168.1.50/24]
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Apply changes:

```
sudo netplan apply
```

5. Hostname Management

The hostname identifies the system on a network.

View Hostname

```
hostname
```

Change Hostname

```
sudo hostnamectl set-hostname myserver
```

6. DNS Configuration

Domain Name System (DNS) resolves hostnames to IP addresses.

DNS Settings Location

```
cat /etc/resolv.conf
```

Example:

```
nameserver 8.8.8.8  
nameserver 1.1.1.1
```

7. Connectivity Testing

Tools for diagnosing network connectivity:

Ping

Checks reachability:

```
ping 8.8.8.8
```

Traceroute

Tracks packet path across networks:

```
traceroute google.com
```

DNS Lookup


```
nslookup google.com
dig google.com
```

8. Routing

The kernel routing table controls how packets are forwarded.

View Routing Table

```
ip route show
```

Add a Static Route

```
sudo ip route add 10.0.2.0/24 via 192.168.1.1
```

Default Gateway

```
ip route | grep default
```

9. Ports and Services

Check Listening Ports

```
ss -tuln
```

Explanation

- -t: TCP ports
- -u: UDP ports
- -l: Listening services
- -n: Numeric format

10. Firewall Management

UFW (Ubuntu)

```
sudo ufw status
sudo ufw allow 22
sudo ufw enable
```

firewalld (RHEL/CentOS)

```
sudo firewall-cmd --list-all
sudo firewall-cmd --add-service=http --permanent
sudo firewall-cmd --reload
```

11. Network Services Management

Networking services may be managed with `systemd`.

Restart Network Manager

```
sudo systemctl restart NetworkManager
```

Check Status

```
systemctl status NetworkManager
```

12. Network Troubleshooting Workflow

- 1. **Interface status**
- 2. `ip link show`
- 3. **IP address**
- 4. `ip addr show`
- 5. **Default gateway**
- 6. `ip route show`
- 7. **DNS resolution**
- 8. `nslookup google.com`
- 9. **Port listening**
- 10. `ss -tuln`

Key Concepts Summary

Concept	Purpose
ip	Primary tool to manage networking
DHCP	Dynamic assignment of IP
Static IP	Fixed IP config for servers
DNS	Resolves domain names

Routing	Directs network traffic
Firewall	Secures network access

Further Reading (Pro Level)

- Linux Networking Architecture
- TCP/IP Stack Deep Dive
- Netfilter + iptables/nftables
- Network performance tuning (`sysctl`)
- VPN and Secure Networking

1. Basic Connectivity

```
ping google.com  
ping -c 4 8.8.8.8
```

2. Interface & IP Details

```
ip a  
ip link  
ip addr show eth0
```

3. Routing Table

```
ip route  
route -n
```

4. Check Open Ports & Services

```
ss -tuln  
ss -tnp  
netstat -tulnp
```

5. DNS Lookup

```
nslookup domain.com  
dig domain.com  
host domain.com
```

6. Test Remote Connectivity

```
telnet IP PORT  
nc -zv IP PORT
```

7. Transfer Files

```
scp file user@server:/path/  
scp -r folder user@server:/backup
```

8. Remote Login

```
ssh user@server  
ssh -i key.pem user@server
```

Subnet Calculation (IPv4 Subnetting)

1. Introduction

Subnetting is the process of dividing a large IP network into **smaller logical networks (subnets)**.

It improves:

- Network performance
- Security
- IP address utilization
- Traffic management

Subnetting is fundamental to:

- Linux networking
- Cloud VPC design
- Data centers
- Routing and firewall configuration

2. IPv4 Address Basics

An IPv4 address:

- Is **32 bits**
- Written in **dotted-decimal format**
- Example:

192.168.1.10

Binary representation:

11000000.10101000.00000001.00001010

3. IP Address Classes (Legacy Concept)

Class	Range	Default Mask
A	1.0.0.0 – 126.0.0.0	255.0.0.0 (/8)
B	128.0.0.0 – 191.255.0.0	255.255.0.0 (/16)
C	192.0.0.0 – 223.255.255.0	255.255.255.0 (/24)

Modern networks use **CIDR**, not classful addressing.

4. Subnet Mask

A **subnet mask** defines:

- Network portion
- Host portion

Example:

255.255.255.0

Binary:

11111111.11111111.11111111.00000000

5. CIDR Notation

CIDR (Classless Inter-Domain Routing) uses slash notation.

CIDR	Mask
/24	255.255.255.0
/25	255.255.255.128
/26	255.255.255.192
/27	255.255.255.224
/28	255.255.255.240
/29	255.255.255.248
/30	255.255.255.252

6. Key Subnetting Formulas

Number of Subnets

$$2^n$$

(where n = borrowed bits)

Number of Hosts per Subnet

$$2^h - 2$$

(where h = host bits)

$-2 \rightarrow$ Network address + Broadcast address

7. Step-by-Step Subnet Calculation Method

Step 1: Identify IP class / CIDR

Step 2: Determine subnet mask

Step 3: Calculate block size

Step 4: Identify:

- Network address
- First usable IP
- Last usable IP
- Broadcast address

8. Block Size Calculation

Block size formula:

$$256 - \text{subnet mask value}$$

Example:

$$/26 \rightarrow 255.255.255.192$$

$$\text{Block size} = 256 - 192 = 64$$

9. Worked Example 1

IP: 192.168.1.10/26

Mask:

255.255.255.192

Block size:

64

Subnets:

0-63
64-127
128-191
192-255

Result (for 192.168.1.10)

Item	Address
Network	192.168.1.0
First Host	192.168.1.1
Last Host	192.168.1.62
Broadcast	192.168.1.63

10. Worked Example 2

IP: 10.0.0.75/28

Mask:

255.255.255.240

Block size:

$256 - 240 = 16$

Subnets:

0-15
16-31
32-47
48-63
64-79

Result

Item	Address
Network	10.0.0.64

First Host	10.0.0.65
Last Host	10.0.0.78
Broadcast	10.0.0.79

11. Hosts per Subnet (Quick Table)

CIDR	Hosts
/24	254
/25	126
/26	62
/27	30
/28	14
/29	6
/30	2

12. Special Subnets

/30

- Used for **point-to-point links**
- 2 usable IPs

/32

- Single host
- Used in routing & loopback

13. Network & Broadcast Address Rules

- First address → **Network ID**
- Last address → **Broadcast**
- Cannot be assigned to hosts

14. Subnetting in Linux (Practical View)

Check IP & mask:

```
ip addr show
```

Routing view:

```
ip route
```


15. Cloud & DevOps Context

Platform	Usage
AWS VPC	CIDR-based subnetting
Docker	Bridge subnets
Kubernetes	Pod CIDR ranges
Firewalls	Rule scoping

16. Common Mistakes

- Forgetting -2 hosts
- Wrong block size
- Mixing classful & CIDR logic
- Assigning network/broadcast IPs

17. Interview & Exam Key Points

- CIDR replaces classful subnetting
- Block size = $2^{56 - \text{mask}}$
- $/26 = 62$ usable hosts
- Network & broadcast are reserved
- Subnetting improves security & efficiency

18. Quick Subnetting Cheat Method

Write subnet mask

Find **interesting octet**

Calculate block size

Locate subnet range

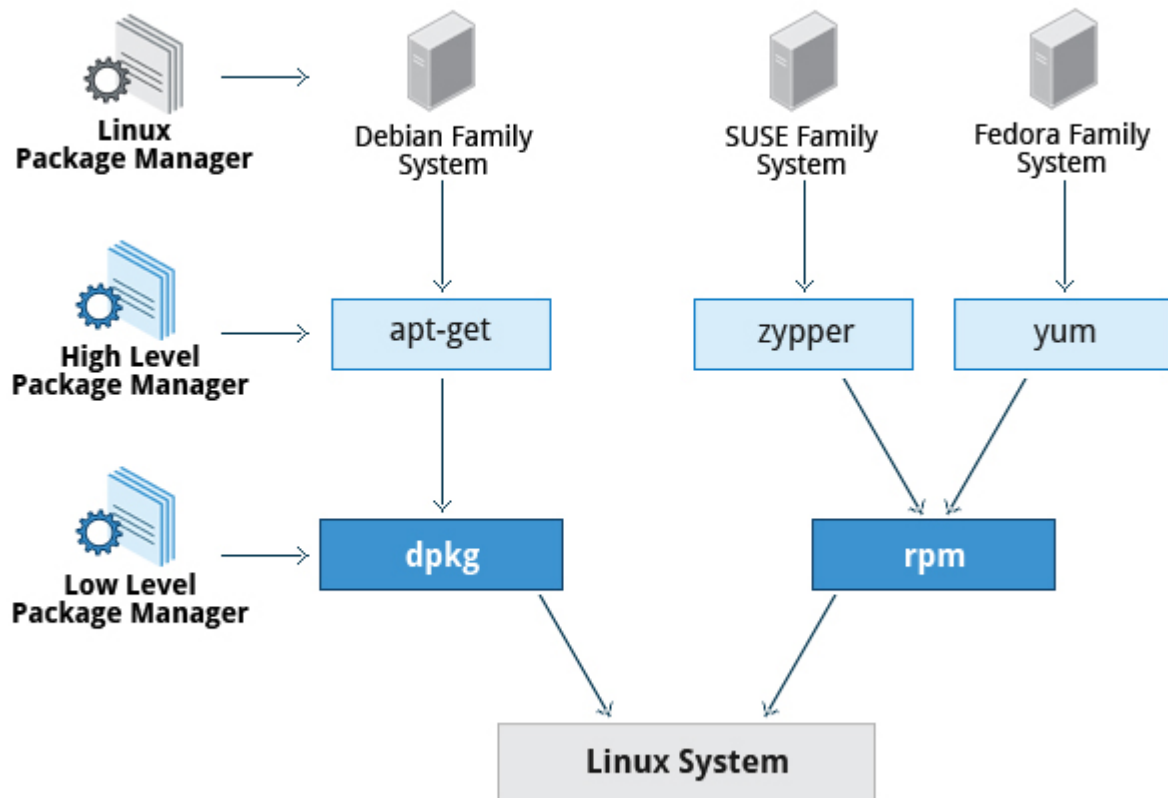
Identify network & broadcast

Practice Problems (Must Do)

1. 172.16.5.20/27
2. 192.168.10.130/25
3. 10.1.1.200/29

Try manually — then verify.

Chapter 9: Package Management



Linux uses different package managers depending on distro.

A. Debian/Ubuntu (APT)

Update Repo & Upgrade

`sudo apt update`

`sudo apt upgrade`

Install / Remove

`sudo apt install package`

`sudo apt remove package`

Search / Info

`apt search nginx`

`apt show nginx`

Clean System

`apt autoremove`

`apt clean`

B. RHEL / CentOS / Fedora (YUM / DNF)

Install / Remove

```
sudo yum install package  
sudo yum remove package
```

```
sudo dnf install package  
sudo dnf remove package
```

Update System

```
sudo yum update  
sudo dnf update
```

Search

```
yum search httpd  
dnf info httpd
```

C. Arch Linux (Pacman)

Install / Remove

```
sudo pacman -S package  
sudo pacman -R package
```

Update System

```
sudo pacman -Syu
```

Search

```
pacman -Ss nginx
```

D. Install from .deb /.rpm

```
sudo dpkg -i file.deb  
sudo rpm -ivh file.rpm
```

E. Snap & Flatpak

```
snap install package  
flatpak install package
```

Chapter 10: Shell Scripting

1. What is Shell Scripting?

- A shell script is a file containing a series of Linux commands.
- Used for automation, backups, monitoring, installation, scheduled jobs, etc.
- Executed by a shell (bash, sh, zsh).

2. Create a Shell Script

nano script.sh

Add this inside:

```
#!/bin/bash
```

```
echo "Hello World"
```

Save → Ctrl + X, Y, Enter

3. Give Execute Permission

```
chmod +x script.sh
```

Run the script:

```
./script.sh
```

4. Basic Script Structure

```
#!/bin/bash
```

```
# This is a comment
```

```
command1
```

```
command2
```

5. Variables

```
name="Tony"
```

```
echo $name
```

User input:

```
read username
```

```
echo "User: $username"
```

6. Command Substitution

```
today=$(date)
```

```
echo $today
```

7. If-Else Conditions

```
if [ $age -gt 18 ]
```

```
then
```

```
    echo "Adult"
```

```
else
    echo "Minor"
fi
Check file:
if [ -f file.txt ]; then echo "File Exists"; fi
```

8. Logical Operators

Operator	Meaning
-eq	equal
-ne	not equal
-gt	greater than
-lt	less than
-ge	≥
-le	≤

File checks:

Flag	Meaning
-f	file exists
-d	directory exists
-r	readable
-w	writable
-x	executable

9. Loops

For Loop

```
for i in 1 2 3
do
    echo $i
done
```

While Loop

```
while [ $num -le 5 ]
do
    echo $num
    num=$((num+1))
done
```

Until Loop

```
until [ $i -gt 5 ]
do
    echo $i
```

```
i=$((i+1))  
done
```

10. Case Statement

```
case $1 in  
start)  
    echo "Starting"  
    ;;  
stop)  
    echo "Stopping"  
    ;;  
*)  
    echo "Invalid"  
    ;;  
esac
```

11. Functions in Shell

```
greet() {  
    echo "Hello, $1"  
}
```

```
greet Tony
```

12. Exit Status

```
echo $?    # shows exit status of last command
```

- 0 → success
- non-zero → error

13. Script Arguments

```
echo $0 # script name  
echo $1 # first argument  
echo $2 # second argument  
echo $@ # all arguments  
echo $# # number of arguments
```

Run:

```
./script.sh arg1 arg2
```

14. Arrays

```
names=("Tony" "Steve" "Natasha")  
echo ${names[1]}
```

```
echo ${names[@]}
```

15. Logging Inside Scripts

```
echo "Backup started at $(date)" >> /var/log/myscript.log
```

16. Scheduling Script Using Crontab

```
crontab -e
```

Example:

```
0 2 * * * /home/user/backup.sh
```

17. Real-World Example Script

Backup Script

```
#!/bin/bash
```

```
src="/home/user/data"
```

```
dest="/backup/data_$(date +%F).tar.gz"
```

```
tar -czf $dest $src
```

```
echo "Backup completed on $(date)"
```